

JAVASCRIPT

```
console.log("Rabindra").
```

extension: → .js

Note: .txt & .js are both source code.

First browser के साथ attach होता था Javascript software.

Except for Node we can also use Deno as Software.

To check installation: node -v.

To run on node: node filename.

⑦ Declaring a Variable

```
const accountID = 144553
```

```
let accountEmail = "hitesh@google.com"
```

```
var accountPassword = "12345"
```

```
accountCity = "Jaipur"
```

console.log (accountID)

⑧ console.table ([])

↳ in array.

gives output as table.



Ex: console.table ([accountID, accountEmail, accountPassword, accountCity]).

(3 ways to declare a variable)

① Const	② let	③ Var	④ Nothing
↳ Will remain constant cannot be reassigned.	↳ Can be changed.	↳ Faces a scope problem	↳ Not a good practice.

(/* Forget */) → Multiline comment.

⇒ Javascript में अगर हम variable declare करेंगे और (let) नहीं तो it is considered as undefined.

⇒ let accountState;

Official Documentation of Javascript.

"tc39.es"

#ecma script.

let age=23 ; let name="Rabindra"
(This is allowed due to semicolon)

otherwise we need to write like this.

let age=23

let name="Rabindra"

① Difference between Undefined and Null

let name ; // name is Khali

let age=null ; // age is not empty rather null

① पहले और अगली के javascript में बहुत फरक है।
कारण javascript के committee ने Decide Kiya to
design an engine that can compile both javascript.
ie old as well as new.

"Use strict" commands tell the engine that all the
code is been written as per new syntax and
methods

* DATA Type Conversion

→ Case study

```
let score = "33"
```

```
console.log (typeof score) ;
```

```
console.log (typeof (score)) ;
```

Data Conversion

```
let valueInNumber = Number (score)
```

```
console.log (typeof valueInNumber)
```

```
console.log (valueInNumber)
```

O/P :->

String	(typeof score)
String	<(typeof (score))>
Number	(value in Number)
33	

Note :->

typeof null is → object

typeof unidentified is → unidentified

Case study 02

```
let score1 = '33ab'
```

```
console.log(typeof score1)
```

```
console.log(typeof (score1))
```

```
let valueInNumber = Number(score1)
```

```
console.log(typeof valueInNumber);
```

```
console.log(valueInNumber);
```

o/p:->

String

String

Number

NaN

Although '33ab' is converted to number

But value is Not a Number (NaN)

Thus we need to be conscious while converting a string to a number, and need to recheck the value.

Suppose we write ① let score1 = "33" → 33

② let score1 = null → ①

③ let score1 = undefined → NaN

④ let score1 = true → 1

⑤ let score1 = false → ①

Boolean Conversion

```
let    var1 = " "  
let    booleanVar1 = Boolean (var1)  
console.log (booleanVar1)
```

O/P: → false

Other case studies

var1 = "rabindra"

O/P: → true

var1 = 0

O/P: → false.

var1 = 1

O/P: → true

OPERATIONS

```
console.log(3*5);
console.log(2+2);
```

O/p :->

15

4

```
console.log((3+4) * 5 % 3);
```

```
let value = 3
let negvalue = -value;
console.log(negvalue);
```

String Addition

```
let str1 = "Rabindra"
let str2 = "Mishra"
let str3 = str2 + str1
console.log(str3);
```



Meme Discussion

```
console.log("1" + 2 + 2)
```

O/p :-> 122

→ अपने आप isne isko
string samaj liya

```
console.log("1" + "2")
```

O/p :-> 12

Rule: If string first aage ke
same string treat honge.

But,

दरता तब तक as Numbers.

```
console.log(2+2 + "3")
```

O/p :-> 43

4 + "3"


```
console.log(true);
```

O/P $\rightarrow$ true.

```
console.log(+true);
```

O/P $\rightarrow$ 1

$\rightarrow$ // Not a good practice

$\rightarrow$ // converts to number

```
console.log(true+);
```

O/P $\rightarrow$ Error.

```
console.log(+"");
```

O/P $\rightarrow$ 0

```
let num1, num2, num3
```

```
num1 = num2 + num3;
```

```
let z = 3
```

```
let x = 3;
```

```
const y = x++;
```

```
const b = ++z
```

$\rightarrow$ Case Study

```
console.log(x, y, z, b)
```

इसमें y का value 3 ही रहे जायेगा

Comparing Different Datatypes

⊙ Problem होता है while comparing different Datatypes

जैसे की :- `console.log("2" > 1);`

Thus we can use typescript.

Note:-> While comparing different datatypes we might not be able to predict our result.

Second example :- `console.log("02" > 1);`

Problematic Comparison

<code>console.log(null > 0)</code>	→ false
<code>console.log(null == 0)</code>	→ false
<code>console.log(null >= 0)</code>	→ true

<code>console.log(undefined == 0)</code>	→ false
<code>console.log(undefined > 0)</code>	→ false
<code>console.log(undefined < 0)</code>	→ false

} It's obvious
undefined के liye
sab false hoga.

Thus avoid all above conversion.



Uniqueness of Javascript:

Strict Comparison. // ===

Difference:

console.log("2" == 2)
o/p: → true

यहाँ पर conversion होता है।
start

console.log("2" === 2)
o/p: → false.

// No conversion

(There are two Categories of Data)

- (1) Primitive
- (2) Non Primitive

Difference between categories is based on how we store the data in memory and how we access it.

➤ Primitive Datatypes

It is basically call by value. ie

Original data ka reference is not given instead we make a copy of original data and then make changes in that object.

7 Types :

→	String
→	Number
→	Boolean
→	null
→	undefined
→	Symbol
→	BigInt.

➤ Non Primitive Datatypes

Array , Objects , Functions

Summary of Datatypes.

```
const score = 75;
const scoreValue = 100.5;
```

```
let userEmail;
const isLoggedIn = false;
const outsideTemp = null;
```

Declaring a Symbol

// These is basically used in React.

Symbol is used with ID's because ID's are unique.

① `const id = Symbol()`

↳ return type is also a symbol

```
const id = Symbol('123')
const anotherId = Symbol('123')
```

(Concept)

Although value is same in both symbol

```
console.log(id === anotherId);
```

o/p: → false

// return value is different.

JS script automatically converts number to bigint.

Non Primitive types

Arrays

```
const heroes = ["Shaktiman", "naagoraj", "doga"]
```

Objects. (key value pairs)

```
{
```

```
  name: "Rabindra"
```

```
  age: 22
```

```
}
```

```
eg: let myobj = {  
      name: "Rabindra",  
      age: 22,  
    }
```

function() : Storing our function as const.

```
const myfunction = function() {  
  console.log("Hello World")  
}
```




Return type of Non Primitive is Object.

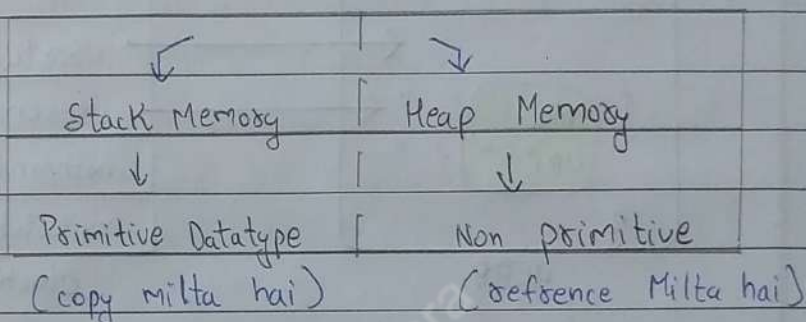
eg:- `console.log (typeof myFunction)`

O/P: $\rightarrow$ function

Although it is returning function but it is Object function

* Memory *

In older languages like C allocating and deallocating the memory used to be our own task. But in modern language we have auto garbage collectors.

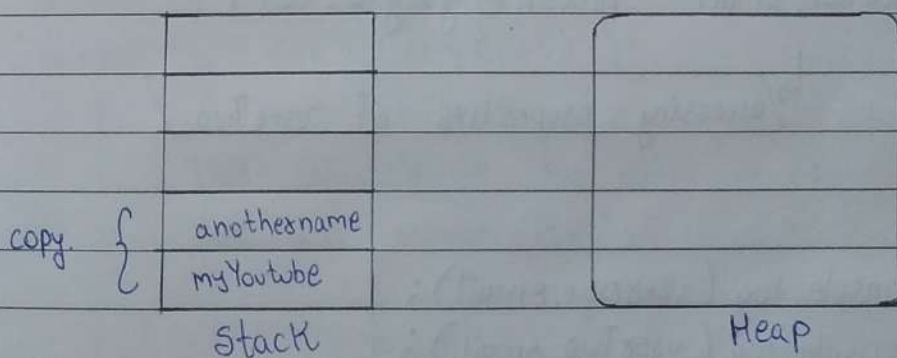


```
let myYoutube = "sabinxaxdotcom"
```

↳ it will be stored in stack.

```
let anothername = myYoutube.
```

```
anothername = "chai aur code"
```



```
console.log ( myYoutube )
```

```
console.log ( anothername )
```

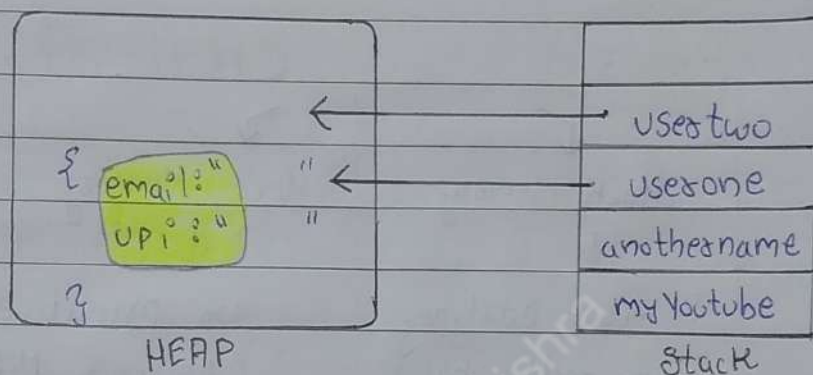
O/P :->

```
sabinxaxdotcom
chai aur code.
```



```
let userOne = {
  email: "user@google.com",
  UPI: "user@ybl"
}
```

↳ Now, this will be stored in Heap



```
let userTwo = userOne
```

Now, userTwo का अलग से एक copy नहीं बन रहा है।

```
userTwo.email = "hitesh@google.com"
```

↳ /accessing properties of userTwo./

```
console.log(userOne.email);
console.log(userTwo.email);
```

∴ Both are Pointing to same reference दोनों में
changes will be displayed

Strings

Q.1) `const name = "Rabindra"`
`const value = 50`

`console.log(name + value);`

o/p → Rabindra50 // However not recommended.

Q.2) How can we declare string as an object?

Ans.

`const str1 = new String("Pokemon")`
// Here, it is declared as an object.

Q.3) Important methods of string?

Ans

`console.log(str.length)`
`console.log(str.toUpperCase());`

→ This will not change the value in original copy but will return it.

Q.4) Finding index based on characters?

Ans.

`console.log(str.indexOf('R'));`

Q.2 Extracting Substrings from strings?

Ans

Suppose,

```
let string = "Rabindra";
console.log(string.substring(0, 4));
```

O/P:→ Rabi {index 0 to 3 included}

Note:→ In this we cannot give negative values then it will consider it from zero.

```
eg: console.log(string.substring(-3, 4));
```

↳ {consider it from (0, 4)}

O/P:→ Rabi

Q.2 Understanding Splice Function ()?

Ans) In string.splice(0, 4) we can also give negative input.

Understanding trim() function.

Ans let var1 = " Rabindra "
 console.log (var1.trim());
 ↳ O/P: "Rabindra"

Replace()

Ans

Let var1 = 'Rabindra Mishra'
 var1.replace(' ', '-')

↳ O/P → 'Rabindra-Mishra'

Q.2 How to convert string object into arrays?

Ans

let var1 = 'Rabindra-Mishra'
 console.log (var1.split('-'));

O/P → ['Rabindra', 'Mishra']

ARRAYS

Collection of multiple items into a single variable

eg:-> `const arr = [0, 4, 33, 'Rabindra'];`

`console.log(arr);`

`console.log(arr[2]);`

O/P:-> `[0, 4, 33, 'Rabindra']`
`[33]`

(2) Method to declare an array

`const arr2 = new Array(1, 2, 3);`
`console.log(arr2.length);`

OPERATIONS () :-

`Push()`

`Pop()`

`Shift()`

`Unshift()`

`Join()`

`slice()`

`splice()`

const arr = [1, 2, 3];

① arr.push(4);

↳ o/p: → [1, 2, 3, 4]

② arr.pop();

↳ o/p: → [1, 2, 3]

③ arr.unshift(5);

↳ o/p: → [5, 1, 2, 3]

// shifts all remaining elements
and adds at 1st index
Thus complexity is high.

④ arr.shift();

↳ o/p: → [1, 2, 3]

// Removes first element.

⑤ arr.indexOf(2);

↳ o/p: 1

⑥ arr.includes(4);

↳ o/p: False.

* * * * * / JOIN() / * * * * *

Combines and joins all elements of an array.

eg: let newArr = arr.join();

↳ o/p: '123'

slice() v/s splice()

```
const arr1 = arr.slice(1, 3)
```

↳ O/P: $\rightarrow [2, 3]$

includes index (1, 2)

But here, original array remains unchanged.
`console.log(arr)` $\Rightarrow [1, 2, 3]$

In case of, splice(): Original arrays get manipulated.

Eg: $\rightarrow$

```
const arr1 = arr.splice(1, 2)
```

↳ O/P: $\rightarrow [2, 3]$

includes (1, 2). index

`console.log(arr)` $\Rightarrow [1]$ ie $[2, 3]$ is removed.

Case study

```
const hero1 = ['Hulk', 'Krish', 'Ironman']
```

```
const hero2 = ['Batman', 'Superman']
```

```
hero1.push(hero2)    # pushing an entire array
```

O/P → ['Hulk', 'Krish', 'Ironman', ['Batman', 'superman']]

Q.2 But we want our required output as.

['Hulk', 'Krish', 'Ironman', 'Batman', 'Superman']?

Ans

i) const allheroes = hero1.concat(hero2)



ii) Using spread operator "..."

```
const allheroes = [...hero1, ...hero2]
```



Q.2 How to check whether something is array or not.

Ans

```
Array.isArray('Rabindra')
```

↳ O/P → False

{It's a string}

Converting Objects into Array

```
const arr = [1, 2, [1, 2], [7, [4, 6]]]
```

// to release the nested structure we can use flat function.

eg:-> `arr.flat(infinity)`

// Not a good practice

↳ `arr.flat(2)`

```
Array.from("Rabindra");
```

↳ O/P: `['R', 'a', 'b', 'i', 'n', 'd', 'r', 'a']`

OBJECTS

const empty = {} creating an empty String/object

const Juser = {

name: 'Rabindra',

age: 23,

hobbies: ['chess', 'cricket'],

[mySym]: "Key",

"gender" : "M"

}

Accessing properties of our objects.

Juser.name	✓	} Both ways
Juser['name']	✓	

But for accessing properties like "gender"
Juser.gender (X) won't work.

Instead Juser["gender"]

● Symbol() (→ it is used to create a symbol value)

const mySym = Symbol("Key2")

↳ for symbol datatype we need to add it in our object [mySym].

Accessing Symbol using / in our Object Jusex.

```
console.log ( Jusex [mySym] )
```

Updating properties of Object
→ Jusex.email = 'mishra'

```
Jusex = {  
  email: 'Mishra',  
  age: 23  
}
```

We can also freeze our Object so that no one can update the object.

→ Object.freeze (Jusex)

String Interpolation

→ (Wrapping our String in backticks.)

Imp

We can also use function () as a variable in Javascript i.e. to store a function in a variable

eg: `user.greeting = function() {
 console.log("Hello friend Utkarsh");
}`

ARRAYS OF OBJECTS

```
const user = [
  {
    email: "rab@123",
    id: 2
  },
  {
    email: "sahulchauchan@03",
    id: 3
  }
]
```

① object

② object

Accessing first object ka email.

```
console.log(user[0].email)
```

↳ O/P: "rab@123"

Case Study 2:

Obj1 = { 1: "a", 2: "b" }

Obj2 = { 3: "a", 4: "b" }

```
console.log(Object.keys(Obj1))
```

↳ O/P: [1, 2]

```
console.log(Object.values(Obj2));
```

↳ O/P: ['a', 'b']

```
console.log(Object.entries(Obj1));
```

↳ O/P: [[1, 'a'], [2, 'b']]

Combining two Objects

```
const obj3 = {obj1, obj2}
```

```
console.log(obj3)
```

// same problem as arrays.

{ obj1: { 1: "a", 2: "b" }, obj2: { 3: "a", 4: "b" } }

Using `assign()` & `"..."` operator

⇒ `obj3 = {...obj1, ...obj2}`

↳ o/p: `{ 1: "a", 2: "b", 3: "a", 4: "b" }`

⇒ `obj3 = Object.assign( {}, obj1, obj2 )`

↳ o/p: `{ 1: "a", 2: "b", 3: "a", 4: "b" }`

Q.2 How to check whether a property is there in an object or not?

Ans `console.log(obj3.hasOwnProperty('name'));`

API: अपना काम किसी और से करवाना!

○ Destructuring the objects

```
const course = {
  coursename: "js",
  price: 100,
  instructor: "Rabindra"
}
```

▣ Syntactical Sugar

```
const {instructor} = course
console.log(instructor)
```

// now we can directly access instructor

In react we can use destructuring this way

eg: const navbar = ({}) => {
}

▣ Suppose, instructor is very long to write we can make it small.

```
const {instructor: ins} = course
console.log(ins)
```

DATE() in JS

Date is arbitrary compared from 1 Jan, 1970
Calculated in millisecond

let myDate = new Date(); console.log (myDate.toString());	// creating an instance of myDate.
--	------------------------------------

Printing it in different methods.

- ① console.log (myDate.toString());
- ② console.log (myDate.toLocaleString());
- ③ console.log (myDate.toJSON());
- ④ console.log (myDate.toISOString());

Imp $\Rightarrow$ (typeof Date) // It is an Object.

● We can also create Arbitrary Date (ie.) Arbitrary Date.

Javascript ^{g.} ~~at~~ Date Month starts with 0

let myDate = new Date (2023, 0, 23)

$\hookrightarrow$ This will create arbitrary date

★ Creating Date as per required format.

eg:

```
let anotherDate = new Date("01-14-2023")  
console.log(anotherDate.toLocaleString());
```

★ Creating current Date() It will be in millisecond.

eg:

```
let myTimeStamp = Date.now()  
console.log(myTimeStamp)
```

↳ O/P: 91723654

(calculated from Jan 1, 1970)

★ myDate.getTime(); // can be used to compare it with previous date.

```
let newDate = new Date();  
console.log(newDate.getMonth() + 1);  
console.log(newDate.getDay());
```

- Locale String (best method)
Locale String ke andar hum object Define Karte hai and bahut saare parameters de sakte hai.

```
newDate.toLocaleString('default', {  
    weekday: "long", // can explore more.  
});
```

FUNCTIONS

① function addtwoNumbers (n1, n2) {
 console.log (n1 + n2);
}

② function SubtracttwoNumbers (n1, n2) {
 return n1 - n2
 // return ke baad kuch nhi
};

③ Printing LoginMessage.

```
function loginUserMessage (username) {  
    return ' $ {username} just logged in'  
}
```

Note: If no argument is given then it will show undefined hence need to handle this case.


```
function login (username) {
```

```
  if (username == undefined) {
```

```
    console.log ('Please enter username');  
  }
```

```
  return `${username} just logged in`  
}
```

● Default Parameters.

```
function loginnew (username = "rabindra") {
```

```
  return `${username} just logged in`  
}
```

If no parameter is given rabindra would be by default

// Suppose, Gift shopping Card.

So in this we not know the numbers we need to add.

▲ "..." Spread / Rest Operators.

depending on use cases we write it as spread like *Kwargs.

IMP

```
(I) function calculatePrice (... num1) {  
    return num1  
}
```

```
console.log (calculatePrice (200, 400, 500));
```

↳ this ...num1 will bundle up all the values [200, 400, 500]

IMP

```
(II) function calculatePrice (val1, val2, ... num1) {  
    return num1  
}
```

```
console.log (calculatePrice (200, 400, 500, 600));
```

In this case num1 will store [500, 600] while val1 will store 200 and val2 will store 400.

➡ We can pass Arguments in two methods

```
const myNewArray = [200, 400, 100]
function returnSecondValue (getArray) {
    return getArray [1]
}
```

passing argument in two ways

- ① returnSecondValue (myNewArray)
- ② returnSecondValue ([200, 400, 500, 1000])

```
const new-user = { // Ensure type safety.
}
```


SCOPE

```
let a=10
const b=20
var c=30
```

```
console.log(a)
console.log(b)
console.log(c)
```

} Here, we will face no Problem while printing it.

जहाँ Problem arises with scope. ie { }

Note: talking about program/function and not Objects

● Var is more problematic.

```
if (true) {
```

```
  let a=10
```

```
  const b=20
```

```
  var c=30
```

```
}
```

```
console.log(a) //Error
```

```
console.log(b) //Error
```

```
console.log(c) o/p:-> 30
```

Explanation: >

console.log(a), console.log(b) will throw error as it is defined in a function and will be out of scope if called outside the function.

But "var" Datatype will print it even outside the function.

Thus we don't use var now.

② let a=100; Global scope

if (true) {

let a=200

const b=50

}

console.log(a);

O/P → 100

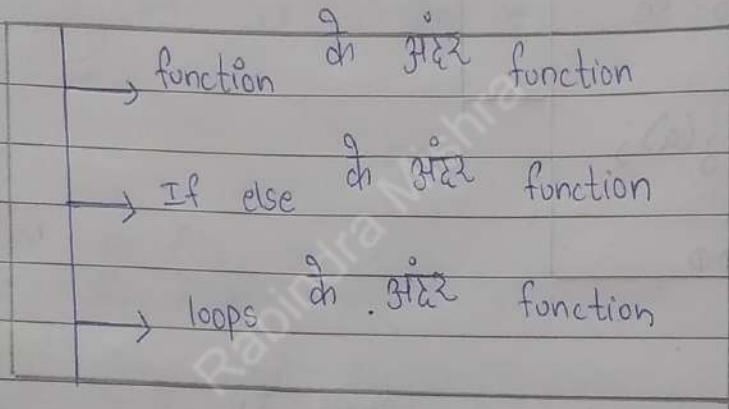
Block scope

Note → through scope अलग है।

SCOPE LEVEL and Mini HOISTING

Global scope can be accessed inside the function but local scope should not be leaked outside

○ Nested scope



Q.2 Find the Output.

```

① function One() {
    const username = "hitesh"

    function two() {
        const website = "youtube"
        console.log(username);
    }

    console.log(website);
    two()
  }
  
```


Solution:->

function One() is parent of two(). Thus two() can access all variables of One(). But one() cannot access two() because then it becomes out of scope.

Output:->

Thus it will give error at console.log(website).

Now, commenting this line. Then two() function will execute and output would be "hitesh"

```
(2) function One() {  
  
    const_name = "rabindra"  
    function two() {  
  
        const website = "youtube"  
        console.log(username);  
    }  
    //two();  
}
```

Now here two() will not execute and thus username would not get printed.

③ if (true) { ①

const username = "Rabindra"

if (username === "Rabindra") { ②

const website = "Youtube"

console.log(username + website);

} ②

console.log(website);

} ①

console.log(username)

O/p: → Rabindra Youtube

It will throw error on ① console.log(website)

② console.log(username)

Because out of scope.

④ function addone(num) { return num + 1 }	const addTwo = function(num) return num + 2 }
--	--

addone(5)

addTwo(5)



⇒ Difference Between these two.

addone(5)

```
function addone(num) {
```

```
  return num+1
```

```
}
```

addTwo(5)

```
const addTwo = function(num) {
```

```
  return num+2
```

```
}
```

→ Executed

→ Error

→ adding/writing/calling it before will not throw any error.

→ But in this it will throw the error. Because it is hoisted in a variable.

ARROW FUNCTION()

This Keyword : Tells about current context

● Registering User

```
const user = {
```

```
  username : 'xab',
```

```
  price : 99
```

```
  welcomeMessage : function () {
```

```
    console.log(`$ {this.username}, welcome to  
    website`);
```

```
  }
```

```
}
```

this Keyword is used to talk about current instance as 'username' will change our message will also change.

Understanding The Concept.

```
user.welcomeMessage()
```

O/p:→ xab, welcome to website

```
user.username = "otkr"
```

// context change.

```
user.welcomeMessage()
```

O/p:→ otkr, welcome to website

ie collective output would be.
 rab, welcome to website
 utk, welcome to website.

② `const user = {`

`username: "Rahul",`

`price: 99,`

`welcomeMessage: function() {`

`console.log(` $ {this.username}, welcome to website`);`

`console.log(this);`

`}`

`}`

`user.welcomeMessage()`

O/P:

Rahul, welcome to website.

{

username: "Rahul",

price: 99,

welcomeMessage: [function]

}



} console.log(this)

Printing Current Context

④ const user = {

username: "Rahul",

price : 99,

WelcomeMessage: function() {

console.log(`\${this.username}, welcome`);

console.log(this);

}

}

user.username

user>WelcomeMessage()

user.username = "RanishK" //context change

user>WelcomeMessage()

O/P:→

Rahul, welcome

{

username: "Rahul",

price : 99

WelcomeMessage: [function]

}

RanishK, welcome //context change

{

username: "RanishK",

price : 99

WelcomeMessage: [function]

}

⑤ const user = {

username : "Rahul",

price : 99,

welcomeMessage : function() {

console.log(`\$ {this.username}, welcome`);

}

}

console.log(this);

O/P:-> {}

It is referring to empty object
but due to node environment

When we will run this on Browser our default
global object is Window object.

* Understanding In Detail *

```
① function chai() {
    console.log(this);
}
```

Whenever we run 'this' inside a function we get many values stored in it.

```
② function chai() {
    let username = "Rab"
    console.log(this.username);
}
```

o/p → Undefined

// We cannot use this inside function

ARROW FUNCTION() Syntax

```
const chai = function () {
    :
}

const chai = () => {
    :
}
```

In arrow function we cannot use this keyword.

① `const addTwo = (num1, num2) => {
 return num1 + num2
 }`

`addTwo(3, 5)`

O/P: 8

We can also do Implicit Return.

② `const addTwo = (num1, num2) => num1 + num2`

③ `const addTwo = (num1, num2) => (num1 + num2)`



If parenthesis likha then
return likha not zaruri

Q.2 How to return a Object?

Solⁿ:

`const addTwo = (num1, num2) => ({username: 'hitesh'})`

`const myArray = [2, 5, 3, 7, 8]`

// Accessing foreach loop.

`myArray.forEach( () => { } )`

OR

`myArray.forEach( () => { } )`

IIFE

➡ Immediately Invoked function Expressions (IIFE)

📌 Used to run certain functions immediately.
Helps in avoiding pollution from global variable.
Can be used for Database Connections.

① function chai() {
 console.log('DB Connected');
}

↓ Making it IIFE

(function chai() {
 console.log('DB Connected');
})(); // we need to put ;

Syntax:

() ()

↳ (function) ↳ (execution call)

chai()

↳ This is called execution

② `(() => {
 console.log ('DB Connected Two');
}) ()`

③ Giving argument in function

`((name) => {
 console.log ('DB connected Two ${name}');
}) ('hitesh')`

④ Named IIFE

`(function chai() { //named IIFE
 console.log ('DB Connected');
}) ();`

CALL STACK

⇒ Javascript execution context

- File kaise execute
- It runs in two phases.

{ } → Global Execution Context

↖ Refered

This keyword.

Browsers and Node # (Global EC) होता है।
Browsers # by default it is window.

↳ Function Execution Context

↳ Eval Execution Context (Mongoose)

{ }

→

Memory Creation Phase

→

Execution Phase



Step 1: Global execution

↓
This

code is run using global execution

```
let val1 = 10
```

```
let val2 = 5
```

```
function addNum(n1, n2) {
```

```
  let total = n1 + n2
```

```
  return total
```

```
}
```

Step 2: Memory Phase

val1 → undefined

val2 → undefined

```
let result1 = addNum(val1, val2)
```

```
let result2 = addNum(10, 2)
```

Memory Phase में सबसे पहले जोया undefined in variable phase.

● First Cycle of Memory Phase.

val1 → Undefined

val2 → Undefined

addNum → definition

result1 → Undefined

result2 → Undefined.

explanation :- addNum is a function it will store definition.

definition will contain

```
{
```

```
  let total = n1 + n2
```

```
  return total
```

```
}
```

● Second Cycle will include execution phase.

Val1 $\leftarrow$ 10

Val2 $\leftarrow$ 5

addnum में kuch execute nhi होगा because iska definition already stored hai.

result1 = addnum

↳ It will create different execution Context

addnum $\rightarrow$	new variable
	Environment
	Execution thread

Sandbox

इसके लिए It will create memory phase and execution phase.

Note: Global EC एक बार define होता है।

● जितने बार addnum use होगा उतने बार Sandbox create होगा



add num →

new variable
environment

← After execution Delete.

Memory Phase

+
Execution thread

val1 → undefined
val2 → undefined
total → undefined

function addNum(n1, n2) {

let total = n1 + n2

return total

}

Execution Context

num1 → 10

num2 → 5

total → 15

return

Global execution context.

For result 2

Memory
PhaseExecution
Phase

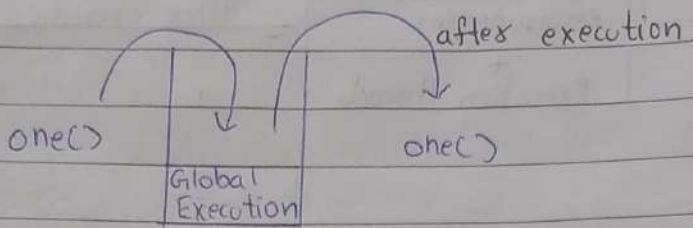
NVE

+

Thread.

Similar, to result 1.

★ Call Stack ★



LIFO : Last in First Out.

Suppose,	one() {		
	two() {	Three()	
	three() {	Two()	
	}	one()	
	}	Global Execution	
}			
}			



"

Basic JS Completed

"

CONTROL FLOW

Everytime full code must not be executed.

Some part of code must run when you are logged In

Some code must run when you are logged out

Thus the need of control flow.

① // if

Syntax:

```
if (condition) {
    :
}
```

This condition must be true.

eg:->

```
const isUserLoggedIn = true;
if (isUserLoggedIn) {
    :
}
```

Conditions can be checked using : $<$, $>$, $<=$, $>=$, $==$, $!=$, $===$, $!==$

eg:->

```
if (2 == "2") {
    console.log("executed");
}
```

$===$: "It will also check Datatype."

(2) 199 ... else

अगर 95 नहीं चलेगा तो सिर्फ else part run की चाहिए

```
eg:-> if (temperature === 41) {
    console.log("Temp < 50");
}
else {
    console.log("Too Hot");
}
```

eg:-> (2)

```
const score = 200
if (score > 100) {
    const power = "fly"
    console.log('user Power : ${power}');
}
console.log('user Power : ${power}');
```

↳ Scope Problem accessing power outside { }

* Short Hand Notation *

Implicit Scope

```
(i) const balance = 1000
    if (balance > 500) console.log("Test");
```

In this automatically scope is considered to be of one line.

```
(ii) const balance = 1000
    if (balance > 500) console.log("Test"), console.log("Test2");
```

↳ writing multiple lines in implicit scope.

(iii)

```
const balance = 1000
```

```
if (balance < 500) {
    console.log("less than 500")
} else if (balance < 750) {
    console.log("less than 750")
} else {
    console.log("> 750")
}
```

//Nested if else

```

③ const debitCard = true
    const loggedIn = true

    if ( loggedIn && debitCard ) {

        console.log (" Shopping allowed ");
    }

```

Switch ()

Syntax:→

```

switch (Key)
{
    case value:
        break;
    default:
        break;
}

```

Note:→ If break statement is not written
 जहाँ की case match हुआ उसके बाद से every
 code line will be executed irrespective to the case
 value except default case.

Truthy Value Concept

Understanding the concept

```
eg:-> const email = "Rabindra@gmail"

if (email) {
  console.log("Got user email");
}
else {
  console.log("Don't have user email");
}
```

(o/p:->) Got user email

concept: Here, it automatically assumes email is true.

```
eg:-> const email = " "

if (email) {
  console.log("Got user email");
}
else {
  console.log("Don't have user email");
}
```

O/p:-> Don't have user email

In this case email is considered as false.

⇒ // List of Falsy Values

false, 0, -0, null, undefined, NaN, "", BigInt 0n

⇒ // Truthy Values (Interview)

"0", 'False', " ", [], { }, function() { }

↳ space

empty function is also truthy value

Q.1 Checking whether an array is empty?

Ans

```
const arr = []
if (arr.length === 0) {
  console.log("Empty Array")
}
```

Q.2 How to check for empty Object?

Ans

```
const emptyObj = {}
```

```
if (Object.keys(emptyObj).length === 0) {
  console.log("Empty Object");
}
```



Nullish Coalescing Operator (??): null undefined.

```
let Val1;
Val1 = 5 ?? 10
console.log (Val1);
```

O/P: → 5

```
Val1 = null ?? 10
console.log (Val1)
O/P: → 10
```

} used for type
Safety

```
Val1 = undefined ?? 15
O/P: → 15
```

**** Interesting Case *****

```
Val1 = null ?? 10 ?? 20
console.log (Val1)
```

O/P: → 10

In this case first non null value will get assigned

TERNIARY OPERATOR

ax:→ condition ? true : false.

```
const iceTeaPrice = 100
```

```
iceTeaPrice >= 80 ? console.log("less than 80") :  
  console.log("more than 80").
```


TERNIARY OPERATOR

Syntax: $\rightarrow$ condition ? true : false.

eg: $\rightarrow$ `const iceTeaPrice = 100`
`iceTeaPrice >= 80 ? console.log("less than 80") :`
`console.log("more than 80").`

FOR LOOP

(H.W)

```
for (let index = 0; index < 6; index++) {  
  const element = index;  
  if (element == 5) {  
    console.log("5 is the best Number");  
  }  
  console.log(element);  
}
```

O/P:->	0
	1
	2
	3
	4
	5 is the best Number

Basic Loop

```
for (let index = 0; index < 2; index++) {  
  console.log(index);  
}
```

O/P:->	0
	1

IMP

⇒ # loop के अंदर loop

```
for (let i = 0; i < 3; i++) {
  console.log(' ${i} Outerloop ');
  for (let j = 0; j < 3; j++) {
    console.log(' ${j} ');
  }
}
```

O/P:->	0 Outerloop
	0
	1
	2
	1 Outerloop
	0
	1
	2
	2 Outerloop
	0
	1
	2

* Imp Keywords *

① Break

(Red card)

② Continue

(एक बार भाफ करो)

Yellow card

➔ Understanding continue;

```
for (let i = 0; i < 4; i++) {
```

```
  if (i == 2) {
```

```
    continue;
  }
```

```
  console.log(i);
```

```
}
```

o/p: → 1 3 4

(2 is skipped due to continue)

⇒ ⇒ While Loop () ⇐ ⇐

```
let index = 0
```

```
while (index <= 10) {
```

```
  console.log('value of index is ' + index);
```

```
  index = index + 2;
```

```
}
```

o/p: → 0 2 4 6 8 10

(Printing only values)

Syntax: → while (condition) {

}

● Applying While loop on myArray ie Array

```
let myarr = ["ironman", "batman"]
let arr = 0
```

```
while (arr < myArray.length) {
  console.log('value is $ {myArray[arr]}');
  arr = arr + 1
}
```

/ Do While loop /

```
let score = 1
do {
  console.log('score is $ {score}');
  score++
} while (score <= 10);
```

Exception of Do While loop.

```
let score = 11
do {
  console.log(score)
  score++
} while (score <= 10)
```

↳ It will run here atleast one time.

High Array Loops in Js

These loops are commonly used for database connection

[" " " "]

[{ }, { }, { }]

Objects inside an Array

① // for of loop

const arr = [1, 2, 3, 4, 5]

for (const iterator of Object) {

↓
Loop applied on object (widescenario)

}

eg:-> for (const num of arr) {

console.log (num);

}

इसमें

we don't have to mention anything ie arr की size it automatically calculates and gives results

● Can also apply on strings

② `const greeting = "Hello World!"`

```
for (const i of greeting) {  
    console.log(i);  
}
```

O/p:->

H
e
l
l
o
:

⇒ // Maps in Javascript (Object)

It holds Key value Pairs.
No Duplicate Values.

eg:-> `const map = new Map()`

`map.set('IN', "India")`

`map.set('SL', "Sri Lanka")`

`map.set('Po', "Portugal")`

// Printing our Map
`console.log(map)`

Applying // for each loop on map.

```
for (const key of map) {  
    console.log(key);  
}
```

O/P:-> It will give output in form of array

['IN', "India"]

['SL', "Sri Lanka"]

['Por', "Portugal"]

* Destructuring Our Array *

```
for (const [key, value] of map) {  
    console.log(key, ':-', value);  
}
```

O/P:-> IN :- India

SL :- Sri Lanka

Por :- Portugal

① Applying for of loop on Object.

```
const myObject = {  
  'game1': 'NFS',  
  'game2': 'GTA',  
}
```

```
for (const [Key, Value] of myObject) {  
  console.log (Key, ':-', Value);  
}
```

AMP

O/p:-> myObject is no iterable.

/ Case 02 /

```
const myObject = {  
  js: "javascript",  
  cpp: "C++",  
  rb: "ruby"  
}
```


For IN loop

PAGE NO.:

① We can apply // for in loop for itexable.

```
for (const Key in myObject) {  
    console.log (Key);  
}
```

// Here, we will only get Key

o/p :-> js
cp
db

// Printing Values of the Objects.

```
for (const Key in myObject) {  
    console.log (myObject [Key]);  
}
```

AMP

② const program = ["js", "db", "py"]
for (const Key in program) {
 console.log (Key);
}

It will print :-> 0, 1, 2, 3 ie (index / Key)

```
(ii) for (const key in program) {  
    console.log ( program[key] );  
}
```

O/P :-> "js", "ab", "Py"

⇒ Applying for in loop on map.

```
const map = new Map()  
map.set ('IN', "India")  
map.set ('Fr', "France")
```

```
for (const key in map) {  
    console.log (key);  
}
```

O/P :-> It will not be executed.

Higher Order Function

PAGE NO.:

```
const coding = ["js", "ruby", "java", "python"]
```

```
coding.forEach ( )  
                  ↳ callback fn
```

callback function Ka naam ही होता है।

It will automatically take parameter of arrays.

```
(i) coding.forEach ( function (val) {  
                          console.log (val);  
                  })
```

```
(ii) coding.forEach ( (item) => {  
                          console.log (item);  
                  })
```

// Arrow function.

We can also give the reference of the function.

```
function printMe (item) {  
          console.log (item);  
}
```

```
coding.forEach ( printMe )
```


forEach can also access multiple things.

```
Coding.forEach ( (item, index, arr) => {
    console.log ( item, index, arr);
})
```

It can also give item, index, arr list.

Arrays are arrays Objects

```
const myCoding = [
    {
        languageName: "JS",
        languageFileName: "Hello"
    },
    {
        languageName: "Python",
        languageFileName: "Org. python"
    }
]
```

```
myCoding.forEach ( (item) => {
    console.log (item, Language Name);
})
```



```
const coding = ["js", "py", "ruby"]
```

```
const values = coding.forEach (item) => {  
  console.log (item);  
}
```

Q.2 What if we store forEach inside a variable?

Ans

```
const values = coding.forEach (item) => {  
  return item  
}
```

```
console.log (values)
```

o/p:-> Undefined // No Values Returned

forEach() does not returns anything.

Note:) forEach() Does not Returns anythings
thus can't be used for conditional return.

● Filter() $\Rightarrow$ This function returns the value

```
const myNoms = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
myNoms.filter ( num)  $\Rightarrow$  num > 4)
```

$\hookrightarrow$ It will return values.

(i)

```
const new = myNoms.filter ( num)  $\Rightarrow$  num > 4)
console.log(new)
```

O/P $\rightarrow$ [5, 6, 7, 8, 9, 10]

(ii)

```
const new = myNoms.filter ( (num)  $\Rightarrow$  {
    num > 4
})
```

O/P $\rightarrow$ It will not print anything because using curly braces { } we have defined scope thus need to use return keyword.

(iii) // Proper Method.

```
const new = myNoms.filter ( (num)  $\Rightarrow$  {
    return num > 4;
})
```


We can also write this using another logic.

eg:->

```
const new = []
myNoms.forEach ( (nom) => {
  if (nom > 4) {
    new.push(nom)
  }
})
```

Quiz Time

Q1 Find what the Function is Doing?

Ans

(i) `const useBooks = books.filter ( (bk) => bk.genre === 'History' )`
`console.log (useBooks)`

(ii) `const book = books.filter ( (bk) => bk.publish >= 2000 )`
`console.log (useBooks)`

(iii) `const b = books.filter ( (bk) => bk.publish >= 2000 & bk.genre === 'History' )`
`console.log (b)`

MAP() $\Rightarrow$ Call back Function. similar to forEach()

eg $\Rightarrow$ `const num = [1, 2, 3, 4]`

```
const newnum = num.map((num) => {return
num + 10})
console.log(newnum)
```

o/p $\Rightarrow$ `[11, 12, 13, 14]`

CHAINING

```
const newNums = myNums.map((n) => n * 10).
map((n) => n + 1)
```

```
console.log(newNums)
```

This is called as chaining

First chain ka result will be passed on to next chain.

other examples $\Rightarrow$

Chaining karta hai

```
const newNum = num.map((number) => number + 1).map(
(number) => number * 2).filter(
(number) => number >= 20)
```

REDUCE()

check on MDN reference for Detail
Mostly used in Debit Card Systems.

Defⁿ:-> reduce() method executes a user-supplied "reducer" callback function.

Demo:

```
const array = [1, 2, 3, 4];
const initialValue = 0;
```

```
const sumWithInitial = array.reduce (
```

```
  (accumulator, currentValue) => accumulator + currentValue,
  initialValue
);
```

```
console.log (sumWithInitial);
```

reduce() :-> we can use accumulator, currentValue.

CurrentValue tells us about currentValue of array.
While accumulator gives us an empty variable.
Initial value will get stored in Accumulator for first iteration.

Accumulator Rn 1st initial value use karega then
It will start storing the value of $accumulator + currentValue$

Thus O/P $\rightarrow$ 10

$\hookrightarrow$ (Dry Run.)

[1, 2, 3, 4]

initial value = 0

Acc + currValue = 0 + 1 = 1

Acc + CV = 1 + 2 = 3

Acc + CV = 3 + 3 = 6

Acc + CV = 6 + 4 = 10

eg $\rightarrow$ 2)

const myNums = [1, 2, 3]

const myTotal = myNums.reduce (function (acc, currVal,

{

return acc + currVal;

}, 0)

$\hookrightarrow$

Initial Value

console.log(myTotal)



● Using Arrow Function

```
const mytotal = myNums.reduce((acc, currval) =>
    acc + currval, 0)
```

```
Q.] const shoppingCast = [
    {
        Name: "JS",
        Price: "3000"
    },
    {
        Name: "Python",
        Price: "500"
    }
]
```

Add all price of shoppingCast?

Ans `shoppingCast.reduce((acc, item) => acc + item.price, 0)`



"

JS

Advance

Basics

Completed"

DOM

- Document Object Model

In HTML web page everything is object

eg:

<head>
<div>
<title>

Founder of JS :-> Brendan Eich

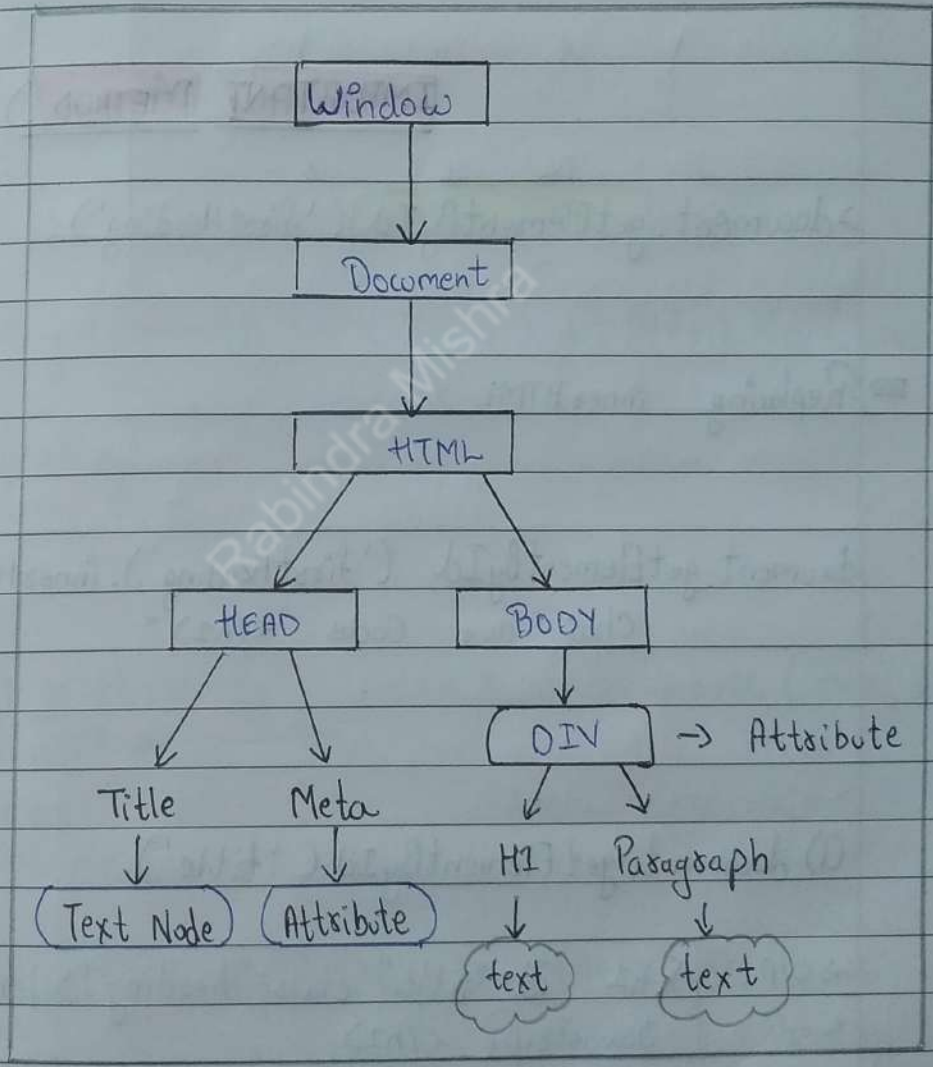
On Browser :-> console.log (window)

→ complex object.

console.log (window.document)

#gt will give html document

Diagram



This is Document Object MODEL

eg: Body is child of HTML and DIV is child of Body.

⇒ console.log(document)

↳ console.log(document.links)

↳ find second link / console.log(document.links)

IMPORTANT METHOD

⇒ document.getElementById('firstHeading');

⇒ Replacing innerHTML

document.getElementById('firstHeading').innerHTML = "<h2>
Chai aur Code </h2>"

(i) document.getElementById('title')

→ O/P: <h1 id="title" class="heading"> learning DOM in
JavaScript </h1>

(ii) document.getElementById('title').id

→ O/P:- 'title'

Will give id value.



> document.getElementById('title').className

O/P: → heading

gives class as output

(iii) document.getElementById('title').getAttribute('id')

O/P: → title

(iv) document.getElementById('title').getAttribute('class')

O/P: → heading.

(v) document.getElementById('title').setAttribute('class', 'test')

O/P: → undefined //gt will overwrite class

ie class = "test"

(vi) document.getElementById('title').setAttribute('class', 'test', 'heading')

O/P: → gt take two values for class ie
class = "test heading"

For easy access we can store it in variable

(i) `const title = document.getElementById('title')`

(ii) style element

(a) `title.style.backgroundColor = "green"`

(b) `title.style.padding = '15px'`

(c) `title.style.borderRadius = '15px'`

(iii)

Adding Content

Q.2 Retrieving text Content.

> `title.textContent`

O/P: → DOM Learning in Javascript

> `title.innerHTML`

O/P: → DOM Learning in Javascript

> `title.innerText`

O/P: → DOM Learning in Javascript



○ textContent v/s innerText

⇒ `<h1 id="title"> Hello
Ram </h1>`

o/p: → as HTML Page

Hello

because Ram is {display: None}

In such cases: (1) title.textContent

o/p: → Hello Ram

(2) title.innerText

o/p: → Hello

● Difference between innerHTML ()

It supports inner HTML tags

eg: → o/p: → `'DOM learning
test `

Query Selectors

(1) `getElementByClass ( )`

(2) `document.querySelector ('h1')`

o/p:→ Will give first h1

eg: `<h1 id="title" class="heading"> ... </h1>`

(3) `document.querySelectorAll ('h1')`

o/p:→ Will give all h1 elements

(4) Selection id / class

(a) `document.querySelector ('#title')`

(b) `document.querySelector ('.heading')`

(5) Selecting Other Elements.

(a) `document.querySelector ('p: first-child')`

(b) `document.querySelector ('input [type="password"]');`



Suppose we have list

one	
two	

```
const myul = document.querySelector('ul')
myul.querySelector('li')
```

↳ It will select first

```
> document.querySelectorAll('li')
```

O/p:-> NodeList(3) [li, li, li]



This is not pure array so don't make beginner mistake.

Output  Node List  

(i) `const templist = document.querySelectorAll('li')`

`> templist`

`o/p: → NodeList(3) [li, li, li]`

Note: Cannot directly apply style to whole array
templist

`eg: → templist.style.color = "green"` //direct (X)

`o/p: → Error`

(b) `templist[0].style.color = "green"`

`o/p: → 'green'`

(ii) `const myH1 = document.querySelectorAll('h1')`

`> myH1`

`o/p: → NodeList [h1 #title.heading]`

Again it is a node list cannot access it directly

`eg: → myH1.style.color = 'green'`

`o/p: → Error`

`myH1[0].style.color = 'green'`

`o/p: → 'green'`



We can also convert NodeList to array and then we can apply methods of Arrays to it.

Iterating on templist.

```
⇒ templist.forEach (function (i) {  
    1. style.backgroundColor = "green";  
})
```

IMP

⇒ HTMLCollection() is different than NodeList.

HTML Collections()

Consider,

<li class "item-list" > One

<li class "item-list" > Two

<li class "item-list" > Three

<li class "item-list" > Four

① const tempClasslist = document.getElementsByClassName('list-item')

> tempclasslist

o/p → HTML Collections()

IMP

> // We cannot Apply forEach loop() on HTML collections

Thus we need to convert it to arrays.

> Array.from(tempclasslist)



storing it into a variable.



```
const myConvertedArray = Array.from(tempClassList)
```

```
> myConvertedArray
```

↳ Now this will point the array.

Now, we can apply `forEach()` loop as we have converted it to an array.

```
eg: → myConvertedArray.forEach (function (li) {
```

```
    li.style.color = "orange"
```

```
})
```

```
myH2.forEach (function (h) {
```

```
    h.style.color = "black";
```

```
    h.style.backgroundColor = "green"
```

```
})
```

Applying multiple properties to H2 elements.

Consider,

```
<div class="parent">
```

```
<div class="day"> Monday </div>
```

```
<div class="day"> Tuesday </div>
```

```
<div class="day"> Wednesday </div>
```

```
<div class="day"> Thursday </div>
```

```
</div>
```

```
<script>
```

```
const parent = document.querySelector('.parent')
```

```
console.log(parent)
```

```
console.log(parent.children)
```

↳ It will give all children of parent

```
eg:-> 0: div.day
```

```
1: div.day
```

```
2: div.day
```

```
3: div.day
```

```
//Accessing first child
```

```
console.log(parent.children[0])
```




Applying for loop to print all values

```
for (let i=0; i<parent.children.length; i++) {
  console.log (parent.children[i].innerHTML);
}
```

o/p:-> Monday

Tuesday

Wednesday

Friday

→ Applying Style on it

```
parent.children[i].style.color = "orange"
```

● Some Important Properties

① console.log (parent.firstElementChild);

↳ Returns first child of Parent

② console.log (parent.lastElementChild);

↳ Returns last child of Parent

eg:-> <div class="day"> Thursday </div>

Imp

We can also move from child (child) → Parent

```
const dayOne = document.querySelector('.day')
```



Concept: querySelector will select first element day class.

Full code in Oneshot:

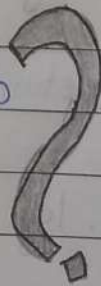
```
const dayOne = document.querySelector('.day')
console.log(dayOne)
```

↳ // Accessing Parent

```
console.log(dayOne.parentElement)
```

Q.} How Does querySelector Works?

Ans It traverses the DOM tree to find out our query.



// Accessing our siblings (next)

Ans console.log(dayOne.nextElementSibling);



`console.log ("Nodes", parent, childNodes);`

- ↳ Will return a complex tree structure of DOM
- ↳ Enter (space) is also considered as text! and is counted in Node List also line breaks.

(2)

```
<body style="background-color: green; color: black;">
```

```
</body>
```

```
<script>
```

```
(i) const div = document.createElement('div')  
    console.log(div)
```

↳ creating a new element.

```
// Adding properties in div element.
```

```
(ii) div.id = "Rabindra"  
      div.className = "main"
```

```
// Setting Custom Attributes
```

```
(iii) div.setAttribute("title", "Generated Title")
```

```
// Adding Styling
```

```
(iv) div.style.backgroundColor = "green"
```

```
// Adding Text in our Div Element.
```

```
(v) div.innerText = "I am learning DOM"
```

↳ But it is not optimized because it first fetches previous reference and then overwrites it.

Thus we can do it in another way.

```
> const text = document.createTextNode("Rabindra")  
div.appendChild(div)
```

Note: → It will not show on our webpage. All this DOM will only be stored in our memory. Thus need to append it.

IMP

```
document.body.appendChild(div)
```

DOM Part 3

PAGE NO.:

● Edit and Remove elements

```
<title> DOM Part 3 </title>
```

```
<body>
```

```
<ul class="language"> <li> JS </li>
```

```
</ul>
```

```
<script>
```

Q.2 Adding a new ?

Ans We can also do this using functions.

```
function addLanguage (langName) {
```

```
  const li = document.createElement('li');
```

```
  li.innerHTML = ` ${langName} `
```

```
  document.querySelector('.language').appendChild(li)
```

```
}
```

↳ selecting our query and appending the child

// Executing our function()

```
addLanguage("Python")
```

```
addLanguage("ML").
```


Optimized way to do this. Because in this it will always traverse the whole tree.

Method 02:

```
function addOptilanguage (langName) {
```

```
    const li = document.createElement('li');
```

```
    li.appendChild(document.createTextNode(langname));
```

```
} document.querySelector('.language').appendChild(li) }
```

● //Edit

(i) `const secondLang = document.querySelector("li:nth-child(2)");`
`secondLang.innerHTML = "Mojo"`

(ii) Method 03:

```
const newli = document.createElement('li')
```

```
newli.textContent = "Mojo"
```

```
secondLang.replaceWith(newli)
```

↳ Taking a reference

(iii) `const secondEdit = document.querySelector("li:first-child")`
`secondEdit.outerHTML = '<li> Typescript </li>'`

↳ In outer HTML you need to give complete HTML query, i.e. tags.



// Remove

ie removing an element

```
const lastlang = document.querySelector('li:last-child')  
lastlang.remove()
```



"DOM COMPLETE."

EVENTS

<script>

- Events Run sequentially in javascript
- Exception : Browser events (mouse, click etc)

</script>

onclick="alert('owl')"

// Not Feasible.

① <script>

```
document.getElementById('owl').onclick = function() {  
    alert('owl clicked');  
}
```

</script>

But this can create a problem because applying onclick on 'id' will or can create problems thus we need to use eventlisteners because we will get propagation

● Method 2: Eventlisteners

- It is a very powerful tool
- Drag n Drop, mouse, click etc.

There are multiple events in eventlistener.
like

(1) click

(2) DoubleClick

(3) Keyboard press etc

Please do visit "MDN" for many More.

Q.2 Interview What is the need of the third Parameter

Ans

Syntax: $\rightarrow$ `document.getElementById('owl').addEventListener('click', function () { }, false);`
By Default.

$\rightarrow$ `document.getElementById('owl').addEventListener('click', function () { alert("owl clicked") } );`

Method (3)

// attachEvent() used to be in "internet explorer."

We don't use this today.

Event Objects

```
document.getElementById('owl').addEventListener('click',  
function (e) {
```

```
    console.log(e);
```

```
});
```

↳ To get all event objects.

O/P:→	which:1
	width:1
	x:125
	y:550
	twist:0
	type:"click"
	target:"img#owl"
	source:
	location:
	⋮

* Important Events *

// type, timestamp, defaultPrevented.

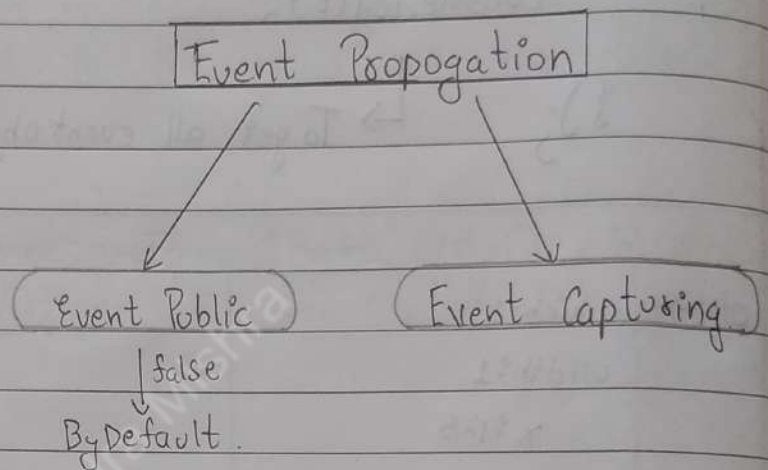
↳ to change default Behaviour
eg: submit Button, anchor tag

// target, toElement, sourceElement, currentTarget,

// altKey, ctrlKey, shiftKey, keyCode.

// clientX, clientY, screenX, screenY.

(2) `document.getElementById('owl').addEventListener('click', function (e) {
 console.log(e);
}, true);`



① Understanding it:

(i) `document.getElementById('images').addEventListener('click', function (e) {
 console.log("click inside ui");
}, false);`

(ii) `document.getElementById('owl').addEventListener('click', function (e) {
 console.log("Owl Clicked");
}, false);`



Understanding above two Code Blocks.

cat

owl

Ball

} We have 3 images under images id.

(i) First eventlistener will print "click inside UI" whenever we click on images

(ii) Second eventlistener will print "Owl clicked" ie specifically for owl. Whenever we will click on owl it will display Owl Clicked

Q.2 While we click on Owl what will the O/P be?

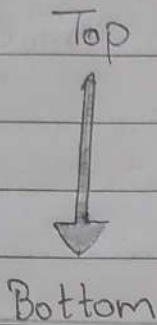
Ans

"Owl Clicked"

"click inside UI"

Principle :- Bubbling up { innermost → outermost }
Public Event Propagation.

Event Capturing



(i) `document.getElementById('images').addEventListener('click', function(e) {
 console.log("click inside UI")
}, true);`

(ii) `document.getElementById('owl').addEventListener('click', function(e) {
 console.log("owl clicked");
}, true);`

o/p: → Click inside UI
owl clicked

} Top to Bottom.

To stop propagation we can use a method.

```
→ document.getElementById('owl').addEventListener('click', function(e)
{
    console.log("owl clicked");
    e.stopPropagation() // gmp
}, false);
```

O/P: → Owl clicked.

// Default Prevented. (To prevent Default Behaviour)

```
① document.getElementById('google').addEventListener
('click', function(e)
{
    e.preventDefault(); // gmp
    console.log("google clicked");
}, false);
```

Now, it will not open google link.

Problem Statement

Q.1 Remove the images on clicking the Elements

Ans

```
document.querySelector('#images').addEventListener('click', function(e) {
```

console.log(e) → It will give events on clicking

console.log(e.target) → It will give the target of the image.

eg: → ``

`e.target.parentNode` → will give `<li> <img> ... </li>`

We have to remove complete `<li>`

Complete Code: →

```
① document.querySelector('#images').addEventListener('click', function(e) {
```

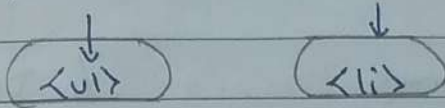
```
  console.log(e.target.parentNode);
```

```
  let removeIt = e.target.parentNode
  removeIt.remove()
```

```
}, false);
```

Method 2: →

removeIt.parentNode.removeChild(removeIt)



ie

	} child
	}

// Problem :

Clicking on google link will remove everything because
 ka parent becomes

Thus requires strict checks.

e.tagName : will print Tag of the event/element.

```
document.querySelector('#.images').addEventListener('click',
function (e) {
```

```
  console.log(e.target.tagName);
```

```
  if (e.target.tagName === 'IMa') { //typescript.
```

```
    console.log(e.target.id);
```

```
    let removeIt = e.target.parentNode
```

```
    removeIt.removeChild
```

```
  }
```

})

Asynchronous Code.

→ Javascript

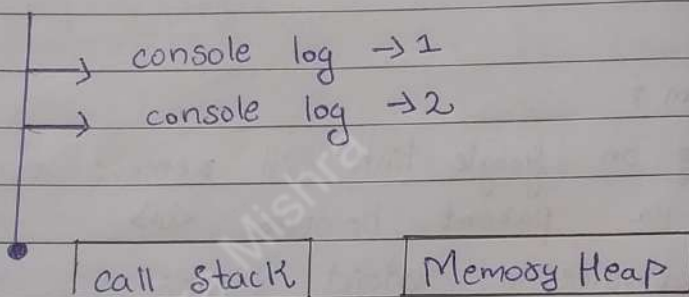
↳ Synchronous

↳ Single Thread

} Default.

→ Execution Context

Ci executes one line of a code at a time.



Each operation waits for the last one to complete before executing.

Node js → Runtime Environment.

● Blocking Code v/s Non Blocking Code

Block the flow of Program ↓ Does not block Execution ↓

Read file Sync.

Read file Async.

eg → sit still till I come

You can do whatever till I come.

Nightmare for any Programming languages.
 because programs cannot read file instead needs
 to give an context to a kernel.

● Reading file requires Node.js.

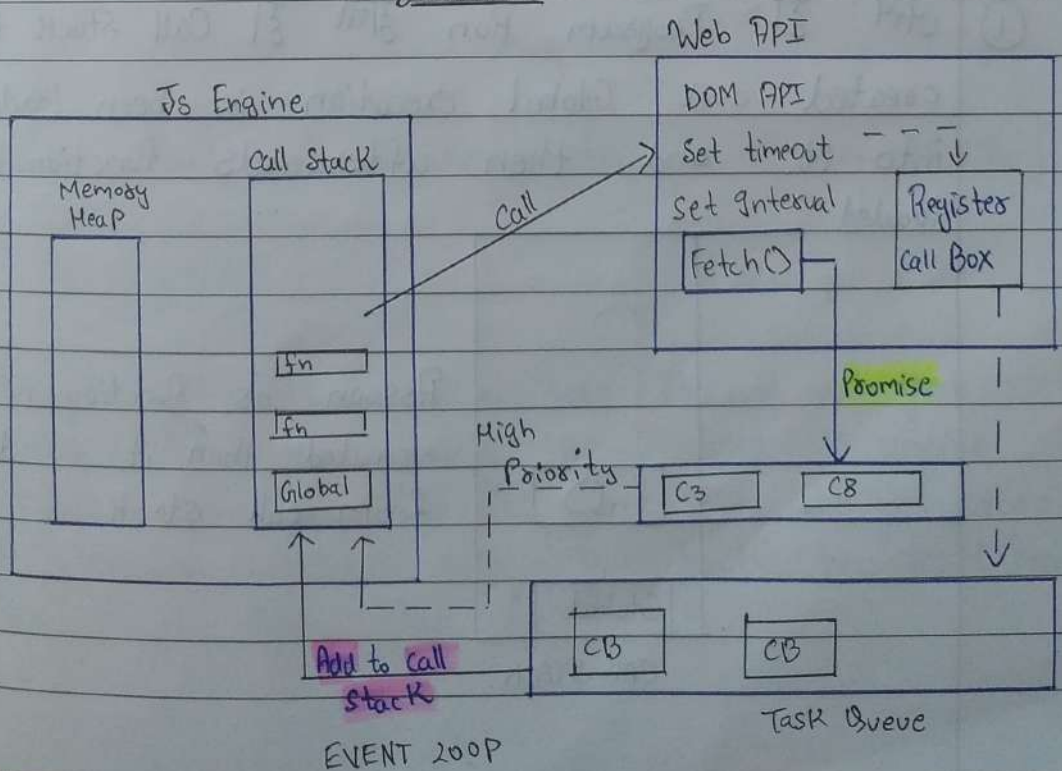
Thus reading file async is better because till the
 time the file is being read by the kernel other
 program lines can execute.

Q.2 Is Asynchronous always Better?

Ans.

Running Database Registration. ie before read/write
 the message could be send to registration.

DIAGRAM



JS Engine बनता है from Memory Heap and Call Stack

JS Engine :-> Memory Heap + Call Stack

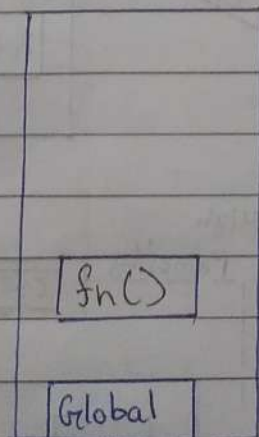
जहाँ से only JS Engine नहीं मिलता है along with that we also get Web API (Browser) में Present होता है।

Only JS Engine: V8

Web API से DOM का access मिलता है।

Task Queue makes JS Asynchronous.
High Priority Queue :-> Promises.

① जैसे ही Program Run होता है Call Stack is created and Global execution is been loaded into it. and then afterwards function is loaded



As soon as function is executed then it is unloaded from call stack.

CALL STACK.

For Asynchronous Operations we have Web API

- DOM API
- set timeout
- set Interval
- fetch()

Set timeout → It timeout function की task is 2sec baad
yaad dila dena, एकी की और 2 min एके
and so on.

Thus we use Register Call Back is used to
register all such events

● STACK: FILO

● NEW API'S in JS : Fetch() comes from
High Priority Queue

" set Timeout की कहानी "

Suppose, [Point 1, Set timeout for 2 of 0seconds, 3]

↳ o/p: → 1 3 2

Although setTimeout for 2 is zero it will not get
printed immediately because then from JS engine
it will call to Web API and then will get store
in Register Call back and push back to task Queue
Thus it will take time for this.

API Requests and V8

API: Talking Point of Two system.

i) frontend. ii) Backend

System can be in different languages as well.

eg: Java $\leftrightarrow$ Javascript.

IMP

APIs ① `api.github.com/users/'name of user'`

↳ अर्थात Profile name.

② `Randomuser.me`

JSON FORMATTER: To make it Readable.

⇒ Important to analyze JSON structure.

* `fetch()` से पहली XML/http request used to be send.
 ↓
 { Heavily AJAX Programming }

We have different state in XML/HTTP like

1) readyState

2) response

3) response text.

⋮

So ON.



Ready State : XML / HTTP

Value	State	Description
0	unsent	Client is created, open() not called yet
1	opened	open() has been called
2	Header_Received	send() has been called
3	Loading	Downloading: ResponseText holds partial Data
4	Done.	The operation is complete.

MDN for : XML/HTTP

Imp

Creating a method of XMLHttpRequest();

thus creating a new method and storing it in a variable.

```
const xhr = new XMLHttpRequest();
xhr.open( Method, url ); // syntax.
```

```
∴ xhr.open( 'GET', "api.github.com" )
```

OR

```
const url = "api.github.com"
xhr.open( 'GET', url )
```

Open अपने आप call nhi होता है।
Thus we need to write.

xhr.send(); send() the request.

(Complete Code Block:)

```
<script>
```

```
const url = api.github.com
```

```
const xhr = new XMLHttpRequest();
```

```
xhr.open('GET', url)
```

```
// Tracking our State
```

```
xhr.onreadystatechange = function() {
```

```
    console.log(xhr.readyState);
```

```
    xhr.send();
```

```
</script>
```

Output of the above request would come in a String. ie response Text.



```
<script>
```

```
const url = "api.github.com"
```

```
const xhr = new XMLHttpRequest();
```

```
xhr.open('GET', url)
```

```
xhr.onreadystatechange = function() {
```

```
  console.log(xhr.readyState);
```

```
  if (xhr.readyState === 4) {
```

```
    const data = this.responseText
```

↳ need to specify context.

```
    console.log(data);
```

```
    console.log(typeof data);
```

```
  }
```

```
} xhr.send
```

```
</script>
```

Note: `typeof data` would be `string`. Because mostly we need to understand that `responseText` is returned as `string`.

Thus, `'data.name'` cannot be accessed because `data` is not an object.

▣ We can convert `string` into `JSON` format and then convert `strings` to `objects`.

eg:-> `const data = JSON.parse(this.responseText).`

Q.2 Document is not a part of core JS?

Ans Yes. It only contains basic arithmetic.

● `Console.log()` is a debugging Tool of browser.

JS runs using C++. Node JS is implementation of V8 engine.

`Console.log()` is there in V8 engine.

V8 is present on OpenSource Github.

PROMISES

Promises object represents the eventual completion or failure of an asynchronous operation and its resulting value.

eg:-> File system, cryptography, Database Request.

Promises future mai complete होता है।

3 States of Promises

- i) Pending : neither fulfilled or rejected
- ii) fulfilled : operation success
- iii) Rejected : operation failed

Mostly promises are consumed but sometimes we also need to create it.

Note: Promise is an Object.

History:-> Q or Bluebird used to be used before promises. These two are libraries. But in Modern Day we don't use it and these two libraries were integrated into Javascript.

Creating a Promise

```
(i) const promiseOne = new Promise()
```

अपनी अंदर रीह
callback leta hai

It has two parts i) Resolve ii) Rejection.

```
const promise = new Promise (function (resolve, reject)
```

3

```
// Do any async Task
```

// DB calls, cryptography, network.

```
setTimeout ( function () { }, 1000);
```

↓ ↓

क्या करना है।	Delay Kitna.
---------------	--------------

eg: `setTimeout ( function() {`

console.log('Async task is complete');

$$\{ 1000 \}$$



Complete Code:->

```
const promise1 = new Promise (function (resolve, reject) {  
  setTimeout ( function () {  
    console.log ('Async task is complete');  
  }, 1000)  
});
```

Consuming the promise

```
promise1.then() {connection with resolve}  
↓  
callback function.
```

```
promise1.then ( function () {  
  console.log ("Promise consumed");  
});
```

But whenever we will run the file the "Promise consumed" would not get printed. Because we would need to connect then() with promise.

```
const promise1 = new Promise ( function (resolve, reject)
```

```
{
```

```
  setTimeout = ( function () {
```

```
    console.log ('Async task is complete');  
    resolve();
```

```
  }, 1000)
```

```
})
```

✓ // Now then is connected.

```
promise1.then ( function() {
```

```
  console.log ("Promise consumed");
```

```
})
```

Output:->

Async task is complete
Promise consumed.

Note:->

then() will always execute after promise



Method 02:

Instead of two parts we can also do it as a single task

```
new Promise (function (resolve, reject) {
```

```
  setTimeout (function () {
    console.log ("Async task 2");
    resolve ();
  }, 1000)
```

```
}).then (function () {
  console.log ("Async 2 resolved");
})
```

then uske se hi attach.

Method 03:

```
const promise3 = new Promise (function (resolve, reject) {
```

```
  setTimeout (function () {
    resolve ( { username: "Rabindra", email: "Hello@" } )
  }, 1000)
```

```
}).
```

We can give/pass data in resolve using object, array etc.

Q. How do we get the data using then().

Ans

```
promise3.then(function (user) {
  console.log(user);
});
```



We have given "user" as a name to our data.

o/p: →

```
{ name: Rabintra , email: 'Hello@' }
```

Method 04

Based on access approved or not eg: error/true

Using Conditional check. (Reject keyword is linked to error).

```
const promise4 = new Promise (function (resolve, reject) {
  setTimeout (function () {
    let error = true
    if (!error) {
      resolve ( { username: 'Mishra', Age: 21 } )
    } else {
      reject ('ERROR: wrong')
    }
  }, 1000)
});
```

Consuming Our promise.

`promise4.then().catch()`

↳ for reject.

जिसना `.then()` लाना है उसमें लगा सकते हैं

Case Study

(i) `promise4.then (user) => {
 console.log (user)
 return user.username
}`

// Arrow function.

- `then()` is returning username where is this exactly going? Let us understand it by storing it in a variable.

IMP

```
const username = promise4.then (user) => {  
    console.log (user);  
    return user.username  
}
```

`console.log (username).`

O/P: → Error

Because this is not the way in which we can collect the data.

In such cases we can use chaining. upar wale `then()` will return it to lower `then()`.

We need to make `let error = false`
otherwise error will get on triggering.

finally () : By default (यह तो होगा ही होगा)

Complete Code Block :

`promise4 . then (user) => {`

`console.log(user)`

`return user.username`

`}`

// will be pass to lower then

`. then (username) => {`

`console.log(username);`

`}`

`. catch (function (err) {`

`console.log(err)`

`}`

`. finally ( ) => {`

`console.log ("Executed Finally")`

`}`

Method 5:



```
const promise5 = new Promise ( function (resolve, reject) {
    setTimeout ( function () {
        let error = true
        if (!error) {
            resolve ( {username: "js", Password: "123"} )
        }
        else {
            reject ('js went wrong')
        }
    }, 1000)
});
```

// semicolons are imp.

- It is not necessary to use `.then()` for promise.
- `async await` can be used.
- Can be used as an alternative although it is good in `catch()`.

```
async function consumePromise5 () {
```

```
    const response = await promise5
    console.log (response)
}
```

// storing await in a variable.

Problem : Cannot handle errors directly.

```
consumePromise5()
```

Thus to resolve this we can use `try: catch:` block.

eg:->

```

async function consumePromise5() {
  try {
    const response = await promise5
    console.log(response);
  }
  catch (error) {
    console.log(error);
  }
}

```

consumePromise5()

```

async function getAllUsers() {

```

// fetch() returns a promise

```

const response = await keyword fetch("httpcode.com/users")

```

// converting our output to JSON

```

const data = response.json()
console.log(data);
}

```

getAllUsers()

↳ But we didn't get error not due to code mistake



Applying try and Catch Block



```

async function getAllUsers () {
  try {
    const response = await fetch(' ')
    const data = await response.json()
    console.log(data);
  }
  catch (error) {
    console.log("E:", error);
  }
}

```

We can also do it using `Fetch()`.

```

① fetch('https://json')
  .then ( response ) => {
    return response.json()
  }
  .then ( data ) => console.log (data)
  .catch ( error ) => console.log (error)

```

Interesting :-> Async complete is running badmaai although code is written first()

FETCH()

- Exciting feature of nodejs
- Changed the way we used to send requests.
- live and interactive Web Pages
- Came in 2022 inside nodejs.

➡ It was heavily based on browser Web Streams API and the AbortController.

Syntax:→

```
fetch("http://example.com/api/endpoint")  
.then((response) => {  
  })
```

```
.catch(function(err) {  
  console.log("Unable to fetch - ", err);  
});
```

Defn: → fetch() method starts the process of fetching a resource from the network, returning a promise which is fulfilled once the response is available.



Q. How to give user-id password or other things?

Ans

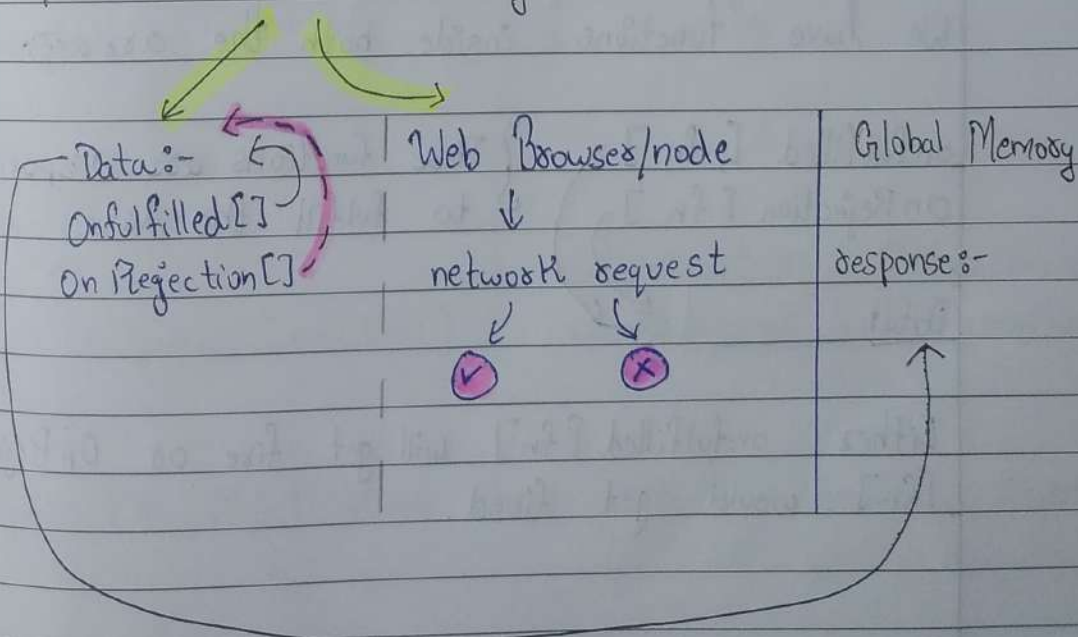
```
eg:-> const headers = {
  'Content-Type': 'text/xml',
  'Breaking-Bad': '<3',
};
fetch('https://', {headers});
```

Please Do use/visit MDN Reference?

Recap:

- `fetch()` Api even `setTimeout` से पहले execute होता है।
Because `fetch()` के लिए special queue बनते हैं।
ie priority Queue (Micro task Queue).

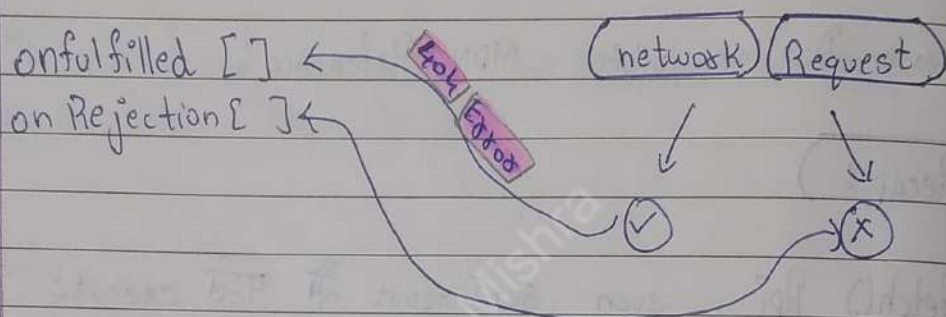
`response = fetch('something')`



`Fetch()` has two parts Data & Web Browser.

onfulfilled []	for Resolution	} Arrays
on Rejection []	for Rejection	

These two arrays are out of Range for us to push something ie they are not in our range.



Note: Even 404 error will be in onfulfilled because only after network request it will get stored in onfulfilled.

We have "functions" inside both the arrays:

onfulfilled [fn]	} These functions are responsible to fulfill the data.
on Rejection [fn]	

Data:

Either onfulfilled [fn] will get fixed or on Rejection [fn] would get fixed.

Once the Data is been completed is given back to Response.

OOP's (OBJECT ORIENTED)

● Javascript and classes

OOP's is an programming style / programming paradigm.

Javascript is a prototype-based language.
It's classes are primarily syntactic sugar.

● Object

- Collection of properties and methods.
- toLowerCase

● parts of OOP

Object literal

Keywords

1) constructor function

2) Prototypes

3) Classes

4) Instances (new, this)

4 Pillars

① Abstraction

② Encapsulation

③ Inheritance

④ Polymorphism

Note: Here, Object literal means it is literally an Object

OOP's in Javascript are just Syntactical Sugar

```
const user = {  
  username: "hitesh",  
  loginCount: 2,  
  signedIn: true,  
  
  getDetails: function() {  
  
    console.log("Details recovered");  
  }  
}
```

Understanding the Objects.

Object name is user.

Properties of user are

- 1) username
- 2) loginCount
- 3) signedIn

Methods of user are

- (a) getDetails

//Printing the value

```
console.log(user.username)
```


OOP's in Javascript are just Syntactical Sugar

```
const user = {  
  username: "hitesh",  
  loginCount: 2,  
  SignedIn: true,  
  
  getDetails: function() {  
  
    console.log("Details recovered");  
  
  }  
}
```

Understanding the Objects.

Object name is user.

Properties of user are

- 1) username
- 2) loginCount
- 3) SignedIn

Methods of user are

- (a) getDetails

//Printing the value

```
console.log(user.username)
```

② `getUserDetails : function () {`

`console.log(this);`
`}`

o/p:-> hitesh

`{`

`username : 'hitesh',`

`loginCount : 8,`

`signedIn : true,`

`getUserDetails : [function : getUserDetails]`

`}`

Purpose :->

Now, again creating it for the second users it will require us to define the Object.

This is why we use "Constructor Function"

Constructor Function

(i) `const promiseOne = new Promise()`

(ii) `const date = new Date()`

Here, `new` keyword is a constructor.

➡ new : This is used to create new instances of same Object().

eg:->

<pre>const date1 = new Date() const date2 = new Date().</pre>

● function User (username, loginCount, isLoggedIn) {
 ↓
 this.username = username;
 ↑
 variable
 this.loginCount = loginCount;
 this.isLoggedIn = isLoggedIn;
 return this
}

Note: ➡ return this function will pass on the entire Object. and will return extra contexts.

```
const userOne = User ("hitesh", 12, true)  
console.log (userOne)
```




PROBLEM:

```
const userOne = User("Rabindra", 23, true)
const userTwo = User("Utkarsh", 30, false)
```

```
console.log(userOne)
```

o/p:→ Utkarsh, logincount: 11, isLoggedIn: false

(i) It is overwriting the value of userOne (userTwo).

Thus we need to use "new" keyword to create a separate instance

```
const userOne = new User("Rabindra", 45, true)
const userTwo = new User("Roy", 36, false)
```

► Important to Understand

(i) Whenever "new" keyword is used an empty object is created which is called as an instance

Step 2: Constructor function is been called up and packs all the arguments and gives us.

Step 3: This keyword gets injected into those arguments.

Step 04: Finally returns it.

```
(i) function User (username, login) {
```

```
    this.username = username;
```

```
    this.login = login;
```

```
    // return this;
```

```
}
```

```
const userOne = new User ("Mishra", true);  
console.log (userOne).
```

Concept: Without writing (return this) also the object will return.

O/P →

```
username : Mishra
```

```
login : true.
```

Note: We can also write methods inside function
eg →

```
this.greeting = function() {
```

```
    console.log ('Welcome & { this.username }');
```

```
}
```



{ Constructor Property }

It is just a reference to itself.

```
console.log (userTwo);
```

```
console.log (userOne.constructor);
```

☆ Visit MDN * for Instance * ☆

JAVASCRIPT

● Magic of Prototype ●

Console of Browser

```
① const newHero = ["HULK", "THOR"]  
    >> newHero
```

```
(2) ['HULK', 'THOR']
```

```
0: "HULK"
```

```
1: "Spiderman"
```

```
length: 2
```

```
• [[prototype]]: Array(0)
```

Behaviour of Javascript is prototypal (Means)

जो Javascript हर चीज जानती है वो उस तक Null value नहीं मिला

Clicking on prototype will open up other several methods like (a) slice (b) sort (c) forEach

Now, this is called as prototypal Behaviour.

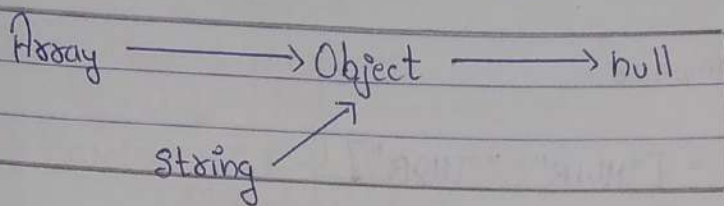
Note: In Javascript each and everything is an object within it.

Concept:

Array $\xrightarrow{\text{is}}$ Object $\xrightarrow{\text{is}}$ null

But Object is not declare from anything (Object Ka no Parent)

Diagram



Explanation :-> Array aur string ka prototype dono hi Object hai, and Object ka null hai.

② Every properties of Object would be available for Array as well as string although it's a different thing that you might not able to use all of them based on access.

③ This is called as inheritance.

BRAINSTORMERS

Note :-> "." access are always with objects.

```
function multiplyby5 (num) {  
    return num*5;  
}
```

// Declaring a Function.

multiplyby5. power = 2

Q.1 Although it's a function can we use it as a object?

Ans: Yes

```
console.log (multiply by 5 (3));
```

```
console.log (multiply by 5 power);
```

```
console.log (multiply by 5. prototype);
```

Output: → 15

2

{}

13 * 5

// null parenthesis indicates multiply by 5 object ka no prototype available.

Javascript mai almost sab cheez ko Object Bana Sakhte hai. Because javascript does not accepts defeat due to its prototypal Behaviour.

After Object → null is End of the Road.

Remember :- this keyword is also attached with prototype

```
function createUser (username, score) {
```

```
  this.username = username
```

```
}
```

".prototype" : By Default some context is set. And by default context is nothing but 'this' of that method.

Prototype also brings along some internal properties with it.

function multiplyby5 (num) { this.num = num return num * 5 }

We always need to take the reference of "this" keyword.

// Understand

create User prototype. increment = 5

Important Concept

We can inject a function inside a prototype of an object

create User prototype. increment = function() { score++ }

But it has a problem i.e. Context Problem
context is not defined ki kiska score increment
karna hai.

eg:→

```
const chai = createUser ("Goodan", 25)
const Buxer = createUser ("Saru", 38)
```

Now in this case it does not know Rishu score must be incremented.

Soln:→

जिसने भी बुलाया है उसकी increment करा दी।
(this)

① create User. prototype. increment = function () {
 this.score ++
}

② create User. prototype. print = function () {

```
// console.log ('score is ${score}'); (x)
```

↳ किसकी score?

```
console.log ('score is ${this.score}');
```

}

Now, here we have injected 2 functions.

i) increment ()

ii) print ()

in javascript we have to just write array.map()
and not array.prototype.map()

This way Js helps us with syntactical sugar.

Similarly, we have to call our injected function.

eg:→ `const chai = createUser ('chai', 25)` (X)



create User

Again, writing it

eg:→	<code>const chai = createUser ('chai', 25)</code> <code>chai.print()</code>	calling injected function.
	<code>o/p:→ { }</code>	

Because, while transferring values from function inside this variable "chai" at that time we didn't mention that we have got some additional properties.

Thus, "new" Keyword is used.

Solution:

```
const chai = (new) createUser ('chai', 25)
chai.print()
```

`o/p:→` Price is 25.

● Recap of `print()` function.

```
const chai = new createUser ('chai', 25).
```

<pre>createUser.prototype.print = function () { console.log ('Price is \$ {score}'); }</pre>
--

{ new Keyword }

→ A new object is created : the new Keyword initiates the creation of a new Javascript Object.

→ A prototype is linked : The newly created Object gets linked to the prototype property of the constructor function. This means that it has access to properties and methods defined on the constructor's prototype.

→ Constructor is called : Constructor function is called with the specified arguments and this is bound to the newly created Object. If no explicit return value is specified from the constructor, Javascript assumes this.

→ new Object is returned : After the constructor function has been called, if it doesn't return a non-primitive value (object, array, function, etc.) the newly created object is returned.

→ Completed :

PROTOTYPE

①

```
let myName = "himanshu M"
```

```
console.log(myName.length)
```

O/P → 12

↳ Included space also

②

```
let myName = "Rab "
```

```
console.log(myName.trim().length)
```

O/P → 3

Method ②

```
console.log(myName.trim().length)
```

Q.2

```
let myName = "Rabindra"
```

```
let mySurname = "Mishra"
```

Create a method to return true length of each string

Ans:

STORY TELLING

Starting our story with Array.

```
let myHeroes = ["thor", "spiderman"]
```

```
let heroPower = {  
  thor: 'hammer',  
  spiderman: 'sling'  
}
```

```
let heroPower = {  
  thor: 'hammer',  
  spiderman: 'sling',
```

```
  getSpiderPower: function () {
```

```
    console.log('spidy power is ${this.spiderman}');  
  }
```

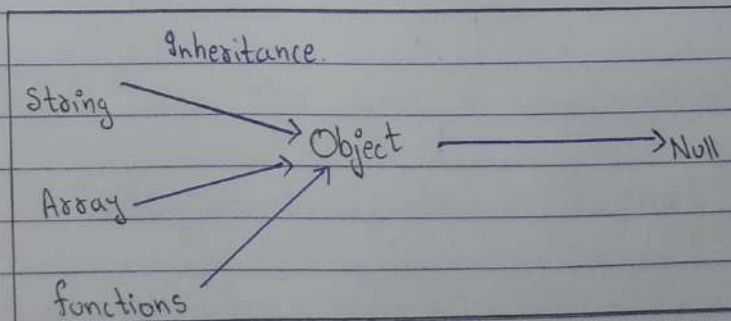
```
}
```

```
Object.prototype.isAbinda = function () {  
  console.log('isAbinda present in all Objects')  
}
```

```
}
```

Concept:

Injecting our function directly in Object



● String, Array or Functions will inherit these function from Object.

① myHeroes.rabindra()

↳ O/P: → rabindra is present in all Objects

Q.2

Let's check whether vice-versa is true or not?

Ans: →

```
Array.prototype.fun = function () {
```

```
  console.log('It's fun');  
}
```

➡ heroPower.fun()

↓

Object par fun() will give error because fun() is injected in an array()

INHERITANCE

(i) `const Teachers = {
 makeVideo : true
}`

(ii) `const TeachingSupport = {
 isAvailable : false
}`

(iii) `const TAsupport = {
 makeAssignment : 'JS assignment',
 fullTime : true
}`

```
> testObj  
  { username : 'hitesh' }
```

⇒ Every Object will have new instances and new properties.

To share information among these 3 objects we can write/ give reference

eg:→ `const TeachingSupport = {
 isAvailable : false
 __proto__ : TAsupport // giving Reference
}`

We can also access it outside Object eg:

`TeachingSupport.__proto__ = TAsupport`

// Modern Syntax (Imp)

Object.setPrototypeOf (TeachingSupport, Teacher)

इसके अंदर

```
let anothername = "Chai aur Code"
```

```
String.prototype.trueLength = function () {
```

```
  console.log (' ${this} ');
```

```
  console.log (' ${this.name} ');
```

```
  console.log (' True length is : ${this.toString().length} ');
```

```
}
```

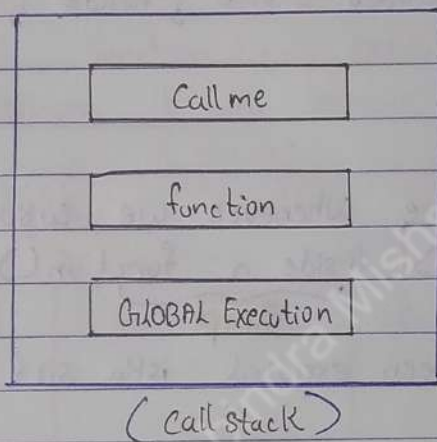

CALL

Problem with "this" Keyword.

```
function () {  
  callme()  
}
```

function di jira function()

So, In such scenarios you won't find to whom this keyword is referring to.



In such scenario "this" represents global execution context i.e. It will refer to window object.

However, Window Object cannot be accessed in Node Environment.

Understanding it with the code.

<pre>function setUsername(username) { this.username = username }</pre>	<pre>function createUser (username, email, password) { setUsername (username) this.email = email this.password = password }</pre>
--	--

O/p:→ email, password would get printed but using function for setting username using a function `SetUsername (username)`.

Thus no name is being printed because "this" keyword is not able to understand the context and is referring window object which is not present in node environment.

```
const rab = new createUser ("rab", "hello@", '123');
```

Problem arises because whenever we write like this `SetUsername (username)` inside a function()

↑
this function is not been executed iska sirf reference pass hogaya hai.

Therefore,
In javascript we have methods to call explicitly such type of functions i.e. `".call()"`

eg:→ `function createUser (username, email, password) {`

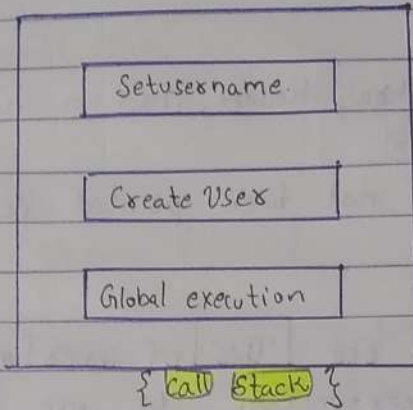
`SetUsername.call (username)`

`this.email = email`

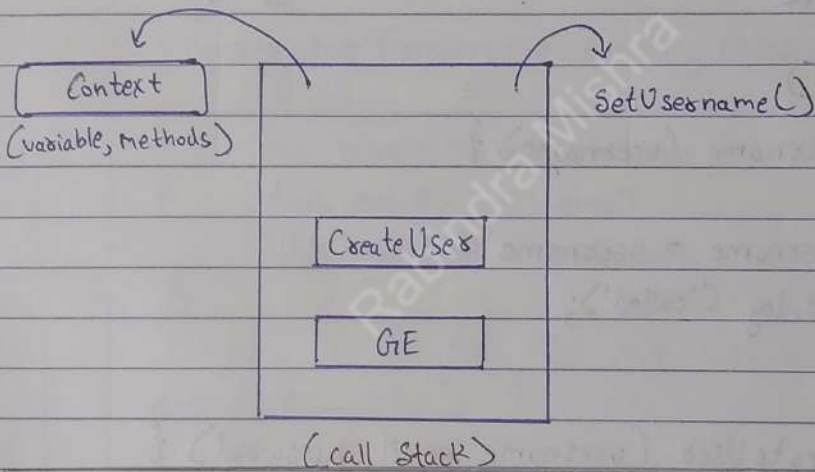
`this.password = password`

Sirf `".call()"` से काम नहीं होता। Because it will only hold reference to execute the function but not the value of the function.

Concept :->



Using call function will execute SetUsername() and after execution it will remove function and its context.



Thus, after execution, the context of SetUsername is removed. Create User won't have its return value.

Eg :->

```
function SetUsername (username) {  
  this.username = username  
  console.log ("Executed");  
}  
  
function createUser (username, email, password) {  
  
  SetUsername.call (username)  
  this.mail = mail  
  this.password = password.  
}
```



```
const fun = new createUser("Mishra", Hello@, "123");
```

O/P:→

Executed

```
createUser { Mishra, Hello@, 123 }
```



● Still setUsername is not been printed i.e. this.username

So here, we will not use 'this' of inner function instead we will say to inner function to use 'this' i.e. context of outer function.

It is telling like मेरे समान अभी आपके हुए!

① Complete Code.

```
function setUsername (username) {
```

```
  this.username = username
```

```
  console.log ('called');
```

```
}
```

```
function createUser (username, email, password) {
```

```
  setUsername.call (this, username)
```

```
  this.email = email
```

```
  this.password = password
```

```
}
```

```
const Rabin@ = new createUser ('chai', "coffe@", "456")  
console.log (Rabin@).
```

↳ O/P:→ Called

```
createUser { name: 'chai', email: 'coffe@', Pass: 456 }
```



(Achievement)

* CLASS CONSTRUCTOR & STATIC *

//ES6

Keyword: Class.

It can contain properties and methods.

```
① class User {  
    constructor();  
}
```

// अपने आप call हो जाता है।
Constructor for class

```
② class User {  
    constructor(username, email, password) {  
        this.username = username;  
        this.email = email;  
        this.password = password;  
    }
```

```
    encryptPassword() {
```

```
        return '$ {this.password}'  
    }
```

```
const chai = new User("chai", "chai@gmail.com", "123")
```

```
console.log(chai.encryptPassword());
```

O/p:→ "123"

⊕ Behind the Scene ⊕

- | |
|----------------------------|
| (i) EK function बनाएगा |
| (ii) Method inject karenge |

```
function User (username, email, password) {  
    this.username = username;  
    this.email = email;  
    this.password = password;  
}
```

// Injecting the function.

```
User.prototype.encrypt Password = function() {  
    return ` ${this.password}`  
}
```

Remember : function can also act as an Object.

INHERITANCE

① Class User {

constructor (username) {

 this.username = username
}

logMe() {

 console.log ('username is \${this.username}');

}

}

// created class successfully.

Class Teacher extends User {

 constructor (username, email, password) {

 this.email = email

 this.password = password

 super (username)

 ↓

this keyword will fetch it from
parent class and will take "this"
of parent class automatically.

 }

addCourse () {

 console.log ('A new course was added by \${this.username}');

}

}

Creating an Object from Class

```
const chai = new Teacher("chai", "chai@teacher", "123");  
chai.addCourse()
```

O/P :-> Proper 😊

```
const masalachai = new User('Masala Chai')  
masalachai.logMe()
```

```
console.log(chai === masalachai);  
console.log(chai === Teacher);
```

Method (2)

```
console.log(chai instanceof Teacher);
```

STATIC PROPERTIES

```
class User {  
    constructor (username) {  
        this.username = username  
    }  
  
    logMe() {  
        console.log ('Username: $ {this.username}');  
    }  
  
    static createId () {  
        return '123'  
    }  
}
```

```
const RabinDra = new User ('rab')  
RabinDra.createId().
```

```
console.log (RabinDra.createId());
```

↳ O/P: Error Because we don't have access of createId

"static" keyword prevents the access of createId()

Q. Can we access it from child.

```
Class Teacher extends User {
```

```
  constructor (username, email) {
```

```
    super (username)
```

```
    this.email = email
```

```
  }
```

```
}
```

Teacher is extending User Class and trying to access static properties.

ie

```
const trying = new Teacher ('Try', 'Try@')
```

```
console.log (trying.createID());
```

↳ O/P :-> Error (No access)

★ Accessing it from the parent class ★

```
class Parent {  
  greet() {  
    console.log('Hello from child');  
  }  
}
```

```
class child extends Parent {  
  fun() {  
    super.greet()  
  }  
}
```

```
const try = new child();  
console.log(try.fun());
```

O/p: 'Hello from child'

OBSERVATION:

constructor is not called so then also it's okay.

BIND in Javascript

● Bind.html

forcefully attach Karo. (सारे this का reference ले पाओ)

Multiple this It is possible that this ka reference खींच गया है।

Thus we need to use Bind() to attach it.

Understanding Bind

<body>

<button> Submit </button>

</body>

<script>

```
class React {
```

```
  constructor () {
```

```
    this.library = "React"
```

```
    this.server = "https://localhost:3000"
```

```
document.querySelector('button')
```

```
  .addEventListener('click', this.handleClick)
```

अब हम इस class call हवा है (class is created)

constructor is defined by using "this" and we have also selected a button.

```
handleClick() {
```

```
  console.log("Button Clicked");
```

```
  console.log(this); console.log(this.server);
```

```
}
```

in this case ↓ (this) (this.server)

↳ show undefined

Because `.addEventListener('click', this.handleClick)`

We are now referring to the context of

```
handleClick () {  
  
}  
}
```

Thus `this.library` and `this.server` will get lost (Ghorm Jayega)

Thus using `this.server` inside `handleClick` will give undefined.
ie.

```
handleClick () {  
  
    console.log (this.server);  
}
```

Solution → `bind()`.

```
.addEventListener ('click', this.handleClick.bind(this));
```

↑

This will bind all this above the line / statement

OBJECTS

Interesting Question { Interview }

```
const mynewObj = { username: "username" }
```

mynewObj

Hidden Object Properties

```
console.log(Math.PI); //universal constant
```

```
Math.PI = 5 //cannot Be overwritten
```

```
console.log(Math.PI) O/P: 3.141
```

getOwnPropertyDescriptors();

```
const descriptor = Object.getOwnPropertyDescriptor(Math, "PI")  
console.log(descriptor)
```

Output :->

```
{  
  value: 3.14159,  
  writable: false,  
  enumerable: false,  
  configurable: false  
}
```

We can also create a Object for eg:->

```
Object.create(null)
```

↳ By Default

```
const newway = Object.create()
```

```
const chai = {  
  name: "chaichat",  
  price: 250,  
  isAvailable: true,  
}
```

```
console.log(chai);
```

We can get property descriptor of this Object chai?

```
console.log(Object.getOwnPropertyDescription(chai));
```

↓
O/P: → undefined

Undefined इसलिए आया Because you asking description of property and not object is "chai"

Soln: →

```
console.log(Object.getOwnPropertyDescriptors(chai, "name"));
```


We can also write/define our Properties.

```
Object.defineProperty (chai, 'name', {  
  writable: false,  
  enumerable: false  
});
```

Now, ∵ Property 'name' writable is turned false
It means cannot overwrite/update this property
in future.

```
console.log (Object.getPrototypeOf (chai, "name"));
```

Iterating an Object is not allowed.

```
for (let [key, value] of chai) {  
  console.log('Execute');  
}
```

↳ o/p: Error

Solution :->

```
for (let [Key, Value] of Object.entries(chai)) {  
    if (typeof value !== 'function') {  
        console.log(`${Key} : ${Value}`);  
    }  
}
```

Security check is used because while iteration function can create problem.

eg:->

```
const chai = {  
    name: 'Mishra',  
    Age : 23  
  
    order: function() {  
        console.log('iteration Problem');  
    }  
}
```

Thus, Objects containing function then iteration would not be suitable without security check.
Although it will not throw error.

Imp

If enumerable : false for a particular property then that property will not get iterated and thus cannot be used/printed during iterations.

GETTER SETTER & STACK

```
Class User {
```

```
  constructor (email, password) {
```

```
    this.email = email ;
```

```
    this.password = password
```

```
  }
```

```
const Ishani = new User ('Ishani@ai', '123')  
console.log (Ishani.password).
```



Simple Execution.

But what if we don't want to show the Password.
Thus we can use getter & setters on property.

Note: getter ke sath hi setter Aata hai

get can be seen as how an object is getting the data. outside the class.

अगर get के साथ set नहीं लिखा तो `console.log ( )`


```
Class User {
```

```
    constructor (email, password) {
```

```
        this.email = email;
```

```
        this.password = password
```

```
    }
```

```
}
```

```
const Ishani = new User ('Ishani @ai', '123')
```

```
Class User {
```

```
    constructor (email, password) {
```

```
        this.email = email;
```

```
        this.password = password
```

```
    }
```

```
    get password () {
```

```
        return this.password.toUpperCase()
```

```
    set password (value) {
```

```
        this.password = value
```

```
    }
```

```
}
```

ERROR :-> Maximum call stack size exceeded.

Error इसलिए आया because continuously we will be trying to set the values thus push call stack is utilized.

(Unique Solution :->)

```
Set password (value) {  
    this.password = value.toUpperCase()  
}  
}
```

↓
thoda sa change Kar do value mai.

OR

We can also change variable name.

```
get Password() {  
    return this._password = value.toUpperCase()  
}
```

∴ get mai agar variable name change hua toh to set() mai bhi change Kar na hai.

```
get Password() {  
    return this._password.toUpper Case()  
}
```

Property ka naam must be same along with get & set Method ().

```
function User(email, password) {  
  this.email = email  
  this.password = password
```

Remember function → function
→ Object

Object.defineProperty (this, 'email', { })

↳ Object

↳ ctrl Property overwrite

↳ Karna Hai

∴ function is also an object thus using these properties we can define the property.
ie `Object.defineProperty()`.

Complete Code:

```
function user (email, password) {  
    this._email = email  
    this._password = password  
      
    Object.defineProperty (this, 'email', {  
        get: function () {  
            return this._email.toUpperCase();  
        },  
        set: function (value) {  
            this._email = value  
        }  
    })  
}
```


Creating Object

```
const JS = new User('chai@chai', 'coffee')  
console.log(chai.email)
```

OBJECTS

```
const User = {  
  email: 'hfchs.com',  
  password: 'abc',  
  get email() {  
    return this._email.toUpperCase()  
  },  
  set email(value) {  
    this._email = value  
  }  
}
```

However, using objects is very rare for get set because syntax looks a bit weird.

(Advantage: Factory function)

Factory function means ^{jo} Object ke Basis par
mai function/object create karke du.

```
const tea = Object.create()
```

↳ Factory function
↳ By Default Null

Understand :->

```
const tea = Object.create(user)  
console.log(tea, email)
```

(Notice) User ke andar "_email" ke property name hai
But still it is giving an output.

O/p :-> 'hfhhs.com'

Because getters and setters ke memory ke upar
rakha hai. thus set() & get() are been used as
a property.

_email :
_password : } Private properties. (-)

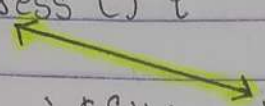
Note: getter and setters ke andar " " ka itna
faisla jahan rakha jata hai.

Maximum call stack size exceed. :- because constructor
gets called continuously.

Race v/s (constructor & get)

Maximum Call Stack Exceed

```
get address () {  
  return `${this.address}`  
}
```



LEXICAL SCOPING &

CLOSURE

Closure is a combination of function bundled together. In other words, a closure gives you access to an outer function's scope from an inner function. at function creation time.

```
(i) function outer() {  
  let username = "Rabindra"  
}  
console.log("Toomuch Outer", username);
```

It's obvious that it will throw the error because we are accessing it out of scope.

Scenario 2:

```
function outer() {  
    let username = "hitesh"  
    function inner() {  
        console.log("hitesh")  
    }  
    inner();  
}
```

// Now it will have access

↓
username

Now inner function has the access.

```
console.log("Too much Outer", username)  
// Output: Throw error
```

In this case both the functions are sharing some Memory Space.

(This is what is called as lexical scoping)

Parent se variable le sakte hai child function but (bhai-bhai) ke variable access nhi kar sakte ho.

Example on next Page.

```
function Outer () {
```

```
  let name = "Thakur",
```

```
    function inner1 () {
```

```
      const age = 21;
```

```
      console.log (name);
```

```
    }
```

```
    function inner2 () {
```

```
      console.log (age);
```

```
    }
```

```
  }
```

Bhai-Bhai share
nahi kar sakte

CLOSURE

Analyzing the Code

```
function makeFunc () {
```

```
  const name = "Mozilla"
```

```
  function displayName () {
```

```
    console.log (name);
```

```
  }
```

```
  return displayName;
```

```
}
```

→ We are returning
displayName
function scope ke
Bahar.

ie (function ka scope hi khatam)

```
const myFunc = makeFunc();  
myFunc();
```

→ यहाँ तो execute होकर
गया from call stack.

→ scope ke Bahar se execute.

Thus there comes the concept of Memory Reference.

Thus. return displayName; Pura lexical scoping is returned instead of that particular function ka executional context.

Understanding Lexical Scoping:

```
function makeFunc() {  
    const name = "Mozilla";  
    function displayName() {  
        console.log(name);  
    }  
    return displayName;  
}  
  
const myFunc = makeFunc();  
myFunc();
```

Execution Context of displayName

lexical scoping of displayName

Thus in this case. name="Mozilla" will get returned in myFunc() and executing displayName(); will now not throw any error.

Closure:

Pura Ka Pura function return hota hai along with "lexical scope"

Use Case:

```
function clickHandler (color) {
```

```
    document.body.style.backgroundColor = `${color}`
```

```
}
```

```
document.getElementById('orange').onclick = clickHandler  
("orange")
```

(Problem:→) onclick expects only function but passing it like this you are directly executing it.

Solution

```
function clickHandler (color) {
```

```
    return function () {
```

```
        document.body.style.backgroundColor = `${color}`
```

```
    }
```

```
}
```

Concept: Due to lexical scoping inner function has access to variable color of clickHandler().